

MSC ARTIFICIAL INTELLIGENCE
MASTER THESIS

Feature-Refined Local Learning Coefficient Estimation

by
BENJAMIN SHAFFREY
13853163

July 4, 2025

48 ECTS
Supervisor:
Leonard Bereska



UNIVERSITEIT VAN AMSTERDAM

Abstract

This thesis proposes and evaluates a method for feature-refined local learning coefficient estimation to study the developmental trajectory of individual neurons in a neural network. Building upon recent work in developmental interpretability and singular learning theory, the research investigates whether sparse autoencoders can extract interpretable features during training, whether data-refined local learning coefficient estimation (drLLC) can identify the specialisation of individual features, and whether the weight-refined local learning coefficient (wrLLC) can track the developmental stages of individual features.

The approach is demonstrated on a toy model of superposition, where sparse autoencoders are trained continuously throughout learning to create “disentangled” models with monosemantic neurons. The method successfully produces disentangled models whose loss curves closely track the original model, and individual neurons do specialize on specific features as confirmed through ablation studies.

However, the research reveals significant methodological challenges that limit the approach’s current utility. The LLC curves for original and disentangled models differ substantially, suggesting either incomplete feature disentanglement or problems with the LLC estimation process itself. More critically, the full model’s geometric complexity does not decompose neatly into individual neurons, and developmental stage boundaries for individual neurons seldom align with those of the complete model.

Three key limitations are identified: imperfect feature disentanglement by sparse autoencoders, convergence and reliability issues with LLC estimation using stochastic gradient Langevin dynamics (SGLD), and arbitrary methodological choices in developmental stage identification. While the conceptual framework shows promise, substantial methodological improvements are needed before feature-refined LLC estimation can reliably provide insights into neural network development.

This work contributes to the broader goal of developing a rigorous scientific understanding of deep learning and neural network development, with implications for AI safety through improved interpretability.

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem	4
1.3	Research Questions	5
1.3.1	Core ideas	5
1.3.2	Training Run for ‘Disentangled’ Model	6
1.3.3	Specialisation of Features	6
1.3.4	Developmental Stages of Features	6
1.3.5	Part-Whole Relationship(s)	7
1.4	Chapter Organisation	8
2	Preliminaries	9
2.1	Multilayer Perceptrons	9
2.2	Transformers	10
2.2.1	Decoder-Only Transformer	10
2.3	Sparse Autoencoders	11
2.4	Statistical Learning Theory	12
2.4.1	Bayesian Statistics	12
2.4.2	Loss Function in Bayesian Statistics	13
2.4.3	Empirical Estimators	14
3	Singular Learning Theory	16
3.1	Singular Models	16
3.2	Free Energy and Local Free Energy	18
3.3	WBIC and RLCT	18
3.3.1	LLC and wrLLC	20
4	Toy Model of Superposition	22
4.1	TMS model	22
4.2	Sparse Autoencoder	23
4.3	Methods	23
4.4	Similarity Scores	23
4.4.1	Mean-max cosine similarity (MMCS)	24
4.4.2	Munkres Algorithm	24
4.4.3	LLC estimation	24
4.4.4	Weight-and-data-refined LLC estimation	24
4.5	Results	25
4.5.1	Training Run for ‘Disentangled’ Model	25
4.5.2	Specialisation of Features	26
4.5.3	Developmental Stages of Features	27
4.5.4	Part-Whole Relationship(s)	28

4.6	Discussion	31
4.6.1	Summary of Main Findings	32
4.6.2	Limitation 1: Imperfect Feature Disentanglement	33
4.6.3	Limitation 2: LLC Estimation Challenges	33
4.6.4	Limitation 3: Developmental Stage Identification	34
4.6.5	Implications and Future Directions	35
5	Conclusion	36
6	Future Work	37
6.1	Multitask Sparse Parity	37
6.1.1	Background and Motivation	37
6.1.2	Task and Model Setup	37
6.1.3	Feature-Refined LLC Estimation Approach	38
6.1.4	Research Questions	38
6.1.5	Expectations	40
6.2	Modular Arithmetic	40
6.2.1	Background and Motivation	40
6.2.2	Task and Model Setup	41
6.2.3	Feature-Refined LLC Estimation	41
6.2.4	Research Questions	42
6.2.5	Expectations	43
6.3	Sorting	43
6.3.1	Background and Motivation	43
6.3.2	Task and Model Setup	44
6.3.3	Feature-Refined LLC Estimation Approach	44
6.3.4	Research Questions	44
6.3.5	Expectations	45
6.4	Greatest Common Divisor	46
6.4.1	Background and Motivation	46
6.4.2	Task and Model Setup	46
6.4.3	Feature-Refined LLC Estimation Approach	46
6.4.4	Research Questions	47
6.4.5	Expectations	48
6.5	Lottery Ticket Hypothesis	48
6.5.1	The Problem	48
6.5.2	Task and Model Setup	49
6.5.3	Feature-Refined LLC Estimation Approach	49
6.5.4	Research Questions	50
6.5.5	Expectations	50
A	Validation TMS	56
B	Validation of LLC convergence	60

Chapter 1

Introduction

Prior work in the field of developmental interpretability has led to the development of an estimator for the local learning coefficient [20], a measure of model complexity that is applicable to strictly singular models, such as neural networks [39]. More recently, this work has been extended to develop a measure of the complexity of a model component, e.g. an attention head in a transformer model [37]. This thesis proposes a reformulation of this latter measure to quantify the complexity of a feature, i.e. a meaningful concept/property of the data that the model has learned to represent.

Circuits, which are thought of as model components performing a particular operation, are composed of such features. The goal of this thesis is to make a contribution to more precisely evaluating the model complexity of a circuit, through being able to evaluate the complexity of features. This may allow us to study when and how circuits form during training. We hope that this work can be built upon to discover methods of detecting circuits in models both during and after training, which may serve as a foothold to later understand more complex structures. We hope that this could give us a wider scope to apply the tools of mechanistic interpretability to obtain a rigorous scientific understanding of neural networks.

1.1 Background

Recent work in the field of developmental interpretability suggests that neural networks may undergo distinct developmental stages that are detectable [18] and that, analogous to biological development [36], this development goes from simple structures to more complex structures across time. This work is built on a mathematical theory known as singular learning theory, developed by Sumio Watanabe [42], [38], [39], [41]. This theory predicts when phase transitions (a concept that comes from statistical mechanics describing a sudden qualitative change in a system’s dynamics) will occur within a dynamical system. Such phase transitions seem to be importantly related to stages of development as proposed in [18] in that they show some correspondence to qualitative changes in model behaviour.

The related field of mechanistic interpretability also seeks to understand the structure present in neural networks but with a focus on trained models. See [1] for an AI-safety-focused review of this field. One of the aims of mechanistic interpretability in particular is to discover the structure present in a neural network in the form of features (directions in activations space) and circuits (connected features in the network that form modular components of the model).

Although there is plenty of work that demonstrates that circuits can be identified in simple models [28] it is in general quite difficult to locate circuits in a trained model. However,

given the above observation that structures develop in neural networks of increasing complexity across time, it seems appropriate to suppose that we may have far more success identifying circuits earlier in the developmental process, perhaps even for reasonably complex models.

1.2 Problem

A key question that then arises is how we can detect the formation of circuits during the developmental process. Recent work suggests that we may apply results from singular learning theory to detect phase transitions not only in the full model but also in a subset of the model’s parameters. These results also seem to suggest that we can discern which subsets of the data are represented by such parameter subsets [37]. This offers the possibility of examining how a certain subset of the parameters in a neural network develops into a circuit or perhaps a component of a circuit.

However, there is an obvious problem with this approach in that it requires us to know which subsets of the parameters will be meaningfully part of the same structure i.e. we need to identify the circuit at training time in order to examine its development. This poses an issue particularly in the case of multilayer perceptron (MLP) layers. Our proposal to circumvent this problem is to focus our attention on the smallest possible model components that we observe to be meaningful and interpretable, namely, a feature.

Of particular interest is the transformer architecture, which forms the core building block of large language models. Although attention layers in a transformer are challenging to interpret, attention-only models are a tractable problem for mechanistic interpretability tools. Progress along these lines has been demonstrated in [14] and [30]. In addition, by treating attention heads in attention-only transformers as model components (circuits), [37] observed both differentiation and specialisation during training: attention heads learn to solve different tasks and become specialised for specific tasks.

Of course, this work can likely be scaled to transformer models with additional attention layers. But the obvious problem is that this does not clearly allow for the inclusion of MLP layers in the transformer architecture. As these layers are instrumental in achieving the ‘emergent’ capabilities when scaled up that LLMs have come to be known for [43], this presents a serious obstacle to scaling up the approach in [37].

Unfortunately, MLPs are quite difficult to make interpretable because many neurons in the hidden layer(s) of an MLP are ‘polysemantic’ i.e. they respond to more than one feature present in the input. As a result, much work in mechanistic interpretability has been devoted towards achieving this aim of making MLPs interpretable, with a particular emphasis on making neurons in the hidden layers of an MLP ‘monosemantic’ i.e. responding to one feature only [12], [19], [2], [33].

A possible explanation for the presence of ‘polysemantic’ neurons in MLPs, given in [13], suggests that neural networks may represent more features than they have dimensions by exploiting feature sparsity in the data. The core idea, known as the ‘superposition’ hypothesis is that a neural network with fewer neurons in its hidden layer(s) than features present in the input is noisily simulating a larger neural network with one neuron per feature by representing more than one feature per neuron. The conjecture is that this is made possible by the sparsity of the features in the input, i.e. it is rare for a neuron representing two features to have to fire in response to both of them, hence this kind of error does not exact a huge cost during training.

Fortunately, much work has been done on taking features out of superposition such as in [2], [33] by applying a sparse autoencoder (SAE) to learn the features present in the hidden layer activations of an MLP.

This leads to the following idea: If we were able to obtain a transformer model where all of the neurons in the MLP layer(s) were ‘monosemantic’, we would then have access to the components of the circuits in the MLP layer, namely, the features. Identifying how these neurons form the components of a circuit within the trained model would still be an intractable task. However, examining their developmental trajectory during training, by detecting phase transitions, would make it easier to identify circuits that form during the developmental process *in the MLP layers of a transformer model*.

Another way of seeing this idea is that we would like to reparameterise model complexity from being in terms of the local learning coefficient (LLC, see [20]), a mapping from weight space to $\mathbb{R}_{\geq 0}$, to an equivalent form that is a mapping from feature space to $\mathbb{R}_{\geq 0}$. In other words, we would like to measure the complexity of features during training rather than weights and use this as a basis to measure the complexity of circuits. In this sense, the approach of weight-refined LLC (wrLLC) estimation in [37] can already be seen as measuring the complexity of circuits but with the limitation that the circuit has already been identified. We propose a method for applying this work to measure the complexity of features during training. Crucially, this approach can be applied to MLP layers. We refer to this approach of measuring the complexity of features as *feature-refined LLC* estimation (frLLC). Note that this is not a new contribution to the mathematics. Our approach is the application of wrLLC to features discovered by applying an SAE to take learned features out of superposition.

In addition, we seek to discover the specialisation of the monosemantic neurons using data-refined LLC estimation. That is to say, we wish to learn the what the features are that the neurons learn to represent.

1.3 Research Questions

Throughout this thesis we will refer to the ‘disentangled’ model. This describes a hypothetical model that has enough neurons in its hidden layer(s) to represent each feature of the input with exactly one neuron [13]. This is the model we hope to obtain when we take features out of superposition.

The broader problem that we hope to make a contribution to in this work can be stated as: **can we track the formation and development of individual computational components (features and circuits) during neural network training, and how do these component-level developmental stages relate to the overall development of the network?**. In trying to make progress in tackling this problem we will develop the core ideas described in the preceding subsection, which can be operationalised as follows:

1.3.1 Core ideas

1. **Training Run for ‘Disentangled’ Model:** For every epoch (checkpoint) of training, we take features in an MLP layer out of superposition using an SAE. This results in a series of models for each checkpoint in which all neurons in the MLP layers are ‘monosemantic’.

2. **Specialisation of Features:** We then apply data-refined LLC estimation to discover *what* such monosemantic neurons learn.
3. **Developmental Stages of Features:** We apply weight-refined LLC estimation to identify moments in time *when* such monosemantic neurons learn features.
4. **Part-Whole Relationship(s):** Finally, and perhaps most importantly, we investigate the relationship between the features and the whole model.

We will now elaborate on each of these in turn and formulate research questions.

1.3.2 Training Run for ‘Disentangled’ Model

The goal here is to have a training run for a ‘disentangled’ model, i.e. a model that has sufficient capacity to represent features monosemantically. The novel contribution here is to not only apply an SAE to take features out of superposition at the end of training but throughout training so that we may also apply the tools of developmental interpretability on a model that is easier to understand. We do not reinitialise the SAE after each checkpoint in order to maintain consistency of the representation of the disentangled model throughout training.

With this in mind, it becomes crucial to evaluate how well such features have been taken out of superposition. Of course, when we know what the features are (as in the toy model of superposition case [13]) we have many ways of assessing this but for all but the simplest models this will not hold. We therefore must rely on the idea that two models that produce the same (or similar) learning curves (in terms of loss and/or accuracy) are approximately the same model. This leads to the following research question:

Research Question 1. *Will the loss and/or accuracy curve of the ‘disentangled’ model be the same or similar to that of the original model?*

As we are also interested in studying the developmental trajectory of the ‘disentangled’ model and how it relates to its components, it is also important to check that the curve of the LLC estimation for both models is the same or similar. This leads to the following research question:

Research Question 2. *Will the LLC curve of the ‘disentangled’ model be the same or similar to that of the original model?*

1.3.3 Specialisation of Features

The goal of applying data-refined LLC estimation is to better understand which feature of the data monosemantic neurons are learning and to take this during the development of the ‘disentangled’ model. This leads to the following research question:

Research Question 3. *Can we discover on which subset(s) of the data a particular monosemantic neuron performs substantially better than all others?*

1.3.4 Developmental Stages of Features

If we have been successful at taking the features in the original model out of superposition then each neuron in the ‘disentangled’ model will be a monosemantic neuron. And so we may apply wrLLC estimation and then the approach outlined in [18] to identify the developmental stages of each of these neurons. Once the developmental stages have been identified we wish to verify that they truly are developmental stages. In order to assess this we check, for each neuron,

that the stage boundaries correspond to meaningful drops in the loss curves for this neuron. That is, for each neuron in each layer, we will prune away (set to zero) all other neurons in the layer and we will compute the loss curve for this neuron on the data for which it is specialised. This leads to the following research question:

Research Question 4. *Will the boundaries of the developmental stages identified for each neuron correspond to meaningful drops in the loss/accuracy curve of a pruned model tested on the relevant subset of data?*

1.3.5 Part-Whole Relationship(s)

As computing the wrLLC for each neuron would be prohibitively expensive for every neuron in a big model, it would be enormously beneficial if we could gain insight into which neurons to analyse in more detail from investigating properties of the whole model.

Having access to a developmental analysis of both a full model and its components offers the potential to discover whether there are detectable relationships between the two. Could it be the case that there are geometric relationships between the whole and the parts that we could understand?

For example, the LLC has an interpretation as “scaling exponent of the volume of ‘nearly’ true parameters” [5]. That is to say that, if λ is the LLC we have a volume function $V(\epsilon)$ for a neighbourhood of parameter with Kullback-Leibler (KL) divergence of less than ϵ that is proportional to:

$$V(\epsilon) \propto \frac{\epsilon^{\frac{\lambda}{2}}}{\sqrt{c_1, \dots, c_{d'}}}, \quad (1.1)$$

(see Section 3.3 for a full explanation of this interpretation and 2.4.2 for an explanation of KL-divergence).

If we succeed at taking features out of superposition, we can think of the neurons as monosemantic. This implies independence between neurons within a given layer. Given this and the interpretation of the LLC above, if we would multiply together an analogous volume function to the one described above for each of the monosemantic neurons, we would obtain the volume of the whole. This would be equivalent to computing the LLCs for each of the neurons and summing these LLCs together (e.g. $\epsilon^{\lambda_1} \times \epsilon^{\lambda_2} = \epsilon^{\lambda_1 + \lambda_2}$). If the above assumptions hold, we would expect to see the same or a similar value for the sum of the LLCs as that for the LLC of the whole model (note that this condition of perfectly monosemantic neurons relates to the case of minimally singular models [5]). This leads to the following research question:

Research Question 5. *If we take the element-wise sum of the LLC curves for each neuron, will the resulting curve be the same or similar to the LLC curve of the entire model?*

As well as this possible geometric relationship it would be instructive to ascertain if there may also be temporal relationships between the neurons and the whole model. We could think of this in terms of the temporal proximity of the boundaries of developmental stages of each neuron and those of the full model. It would be hugely beneficial to know if the boundaries of the individual neurons coincide with those of the full model. If this were the case, we would be able to see in the analysis of the full model that the stage boundaries indicate a moment where one or more neurons are undergoing a transition which in turn could inform us on how to identify these neurons. This leads to the following research question:

Research Question 6. *For each neuron and for each of the boundaries of its developmental stages, is there a boundary of a developmental stage in the full model with which it coincides?*

The ultimate goal of this work is to leverage knowledge of the local geometry of the loss landscape when training the full model to detect moments in the training when circuits form. This would be a significant step toward understanding how structures form in neural network weights, allowing detailed understand of early structures that could inform our approach to later, more complex structures and perhaps even fully trained models.

1.4 Chapter Organisation

The remainder of the thesis is organised as follows: Chapter 2 provides a brief overview of important preliminary topics and concepts related to this work. Chapter 3 gives a brief overview of important concepts in Singular Learning Theory. Then to answer the research questions described above we will run a series of experiments that will be detailed in Chapter 4: we will train a version of the original TMS model in [13] but as specified in [10]. We will then train an SAE on each of the toy model checkpoints and apply a reparameterisation to obtain a representation of the disentangled model for each of the training checkpoints. We will then apply tools from developmental interpretability to identify the developmental stages in this disentangled model and each of its ‘monosemantic’ neurons and analyse the extent to which these developmental stage boundaries agree/overlap/disagree. Chapter 5 is a conclusion to the work done thus far and Chapter 6 is a detailed proposal for future research work that will build upon this work.

Chapter 2

Preliminaries

The explanations in this section draw extensively on [4], please see more comprehensive explanations there.

2.1 Multilayer Perceptrons

Let $W \subseteq \mathbb{R}^D$ denote the weight space, where D is the total number of weights and biases. A Multilayer Perceptron (MLP) is a function that maps inputs through a series of transformations into a feature space that, it is hoped, has a simple mathematical structure so that a linear model such as linear regression (in the case of regression) or logistic regression (in the case of binary classification) can be applied. The transformations correspond to layers in the MLP and the number of layers is known as the depth L . The widths of the layers indicate the number of neurons in each layer.

So the MLP has depth L with widths $(d_0, \dots, d_L) \in \mathbb{N}^{L+1}$ such that there are $d_0 = N$ inputs and $d_L = M$ outputs, and such that each layer applies an affine transformation followed by an element-wise nonlinear transformation, known as an activation function. The mapping is expressed as

$$f : \mathbb{R}^N \times W \longrightarrow \mathbb{R}^M, \quad (2.1)$$

and the function is defined as

$$f(x, w) = (A_L \circ \sigma \circ A_{L-1} \circ \sigma \circ \dots \circ \sigma \circ A_1)(x) \quad (2.2)$$

such that for each layer $1 \leq l \leq L$ there is an affine function $A_l : \mathbb{R}^{d_{l-1}} \rightarrow \mathbb{R}^{d_l}$, parameterised by weights $w^l \in \mathbb{R}^{d_{l-1} \times d_l}(\mathbb{R})$ and biases $b^l \in \mathbb{R}^{d_l \times 1}(\mathbb{R})$, and where:

- $A_l : \mathbb{R}^{d_{l-1}} \rightarrow \mathbb{R}^{d_l}$ are affine transformations defined by $A_l(x) = W_l x + b_l$
- $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ are weight matrices
- $b_l \in \mathbb{R}^{d_l}$ are bias vectors
- $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a nonlinear activation function applied elementwise
- d_l is the width (number of neurons) of layer l
- $\circ$ denotes function composition

The total number of parameters D is given by:

$$D = \sum_{l=1}^L (d_l \times d_{l-1} + d_l) \quad (2.3)$$

A common choice for σ is the rectified linear unit (ReLU) function, defined as

$$\sigma(x) = \max(0, x), \quad (2.4)$$

which will be the activation function considered in any MLP referred to in this thesis. Along with attention layers, and residual streams, MLP layers are one of the core building blocks of transformer models, upon which highly capable and generalisable large language models (LLMs) such as GPT-4 are built [3].

2.2 Transformers

A transformer is a neural network that can consist of either an encoder or a decoder, or both as in the original paper it was proposed [35]. As the transformers considered in this thesis are decoder-only we will focus our attention on these kinds of transformers.

2.2.1 Decoder-Only Transformer

A decoder-only transformer maps an input sequence (context) to a probability distribution of the next token given the previous tokens, i.e. the input (context). For a given input sequence $x = (x_1, \dots, x_K)$, the transformer applies several transformations consisting of:

Input Embedding

The first step embeds tokens and adds positional information:

$$h^0 = \text{Embed}(x) + \text{PositionalEncoding} \quad (2.5)$$

Transformer Layers

For each layer l from 1 to L :

$$h^l = \text{TransformerLayer}(h^{l-1}) \quad (2.6)$$

Each transformer layer consists of

Multi-head Attention: For each head i from 1 to H , where H is the number of heads, we have:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.7)$$

$$= \text{softmax} \left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}} \right) (VW_i^V) \quad (2.8)$$

The heads are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1; \dots; \text{head}_h)W^O \quad (2.9)$$

MLP: each layer includes a position-wise feed-forward network consisting of two linear transformations with a ReLU activation:

$$\text{MLP}(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2 \quad (2.10)$$

Layer Normalization and Residual Connections: The output of each sub-layer is processed through layer normalization and residual connections, where LayerNorm normalizes activations x across the feature dimension according to

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad (2.11)$$

and the residual connections allow for the addition of the input of the layer unchanged to the output of the layer, i.e.

$$\text{Output at layer}_i = x + \text{layer}_i(x) \quad (2.12)$$

Unembedding (Output: For autoregressive language models, the output probability at position k is:

$$P(x_{k+1}|x_{1:k}) = \text{softmax}(h_k^L W_{\text{out}}) \quad (2.13)$$

2.3 Sparse Autoencoders

An autoencoder is a neural network trained to reconstruct its input at the output layer. Given unlabeled training examples $x^{(1)}, x^{(2)}, x^{(3)}, \dots$, where $x^{(i)} \in \mathbb{R}^n$, an autoencoder sets the target values equal to the inputs, i.e., $y^{(i)} = x^{(i)}$. The network tries to learn a function $h(x) \approx x$, effectively attempting to approximate the identity function [27].

A Sparse Autoencoder (SAE) is an autoencoder with an overcomplete latent layer, i.e. more neurons in its hidden layer than there are neurons in its input layer or output layer. This overcompleteness, along with the application of a sparsity penalty during training allows SAEs to solve the dictionary learning problem [11] by learning a dictionary of vectors

$$\{\mathbf{f}_k\}_{k=1}^{n_{\text{feat}}} \subset \mathbb{R}^d, \quad (2.14)$$

that can represent the unknown ground truth features in the activations

$$\{g_j\}_{j=1}^{n_{\text{gt}}}, \quad (2.15)$$

as sparse linear combinations [1]. As with all autoencoders, the SAE consists of an encoder and a decoder where the encoder is given by

$$\mathbf{h} = \text{ReLU}(W_{\text{enc}} \mathbf{x} + \mathbf{b}) \in \mathbb{R}_{\text{hid}}^d, \quad (2.16)$$

where

$$d_{\text{hid}} = R d_{\text{in}}, \quad (2.17)$$

i.e. the hidden dimensionality is some multiple times the input dimensionality, hence the overcompleteness. The decoder is then given by

$$\mathbf{x}' = W_{\text{dec}} \mathbf{h}, \quad (2.18)$$

where $W_{\text{enc}}, W_{\text{dec}}^T \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{hid}}}$. The parameter matrix W_{dec} represents the dictionary of learned features, with rows $\mathbf{f}_i$ being the features, hence we may rewrite the above as

$$W_{\text{dec}} \mathbf{h} = \sum_{i=0}^{d_{\text{hid}}-1} h_i \mathbf{f}_i. \quad (2.19)$$

What differentiates SAEs from standard autoencoders is not just the overcomplete hidden layer but also the property of sparsity obtained in this hidden layer. This is achieved by adding an L_1 norm penalty term to the loss function of the autoencoder, giving us

$$\mathcal{L}(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}'\|_2^2 + \alpha \|\mathbf{h}\|_1. \quad (2.20)$$

Note that we have the reconstruction error here as the first term, as is standard in the case of dimensionality reduction and the second term is a regularising term, specifically the L_1 norm because this penalty term has the property of shrinking any components of $\mathbf{h}$ to zero unless they contribute significantly to achieving lower reconstruction loss, hence how the sparsity constraint is applied. α is then a hyperparameter that controls the strength of the non-sparsity penalty.

2.4 Statistical Learning Theory

2.4.1 Bayesian Statistics

Consider a probability space $(\Omega, \mathcal{B}, \mathbb{P})$ and a joint random variable $(X, Y) : \Omega \mapsto \mathbb{R}^N \times \mathbb{R}^M$ with distribution $q(y, x) = q(y | x)q(x)$, where X (of dimensionality N) is the input and Y (of dimensionality M) is the output. Our goal is to estimate the true distribution $q(y, x)$ given a dataset D_n of n independent and identically distributed samples of the form

$$D_n = \{(X_1, Y_1), \dots, (X_n, Y_n)\}. \quad (2.21)$$

In the examples that we will consider in this thesis the distribution of inputs $q(x)$ is known. So we seek to learn the distribution of the outputs $q(y | x)$. By the IID assumption, we can write the true joint distribution of this dataset as

$$q(D_n) = q((x_1, y_1), \dots, (x_n, y_n)) = q(x_1, y_1) \cdots q(x_n, y_n). \quad (2.22)$$

We attempt to model this true distribution with a parametric model

$$p(y, x | w) = p(y | x, w)q(x). \quad (2.23)$$

By the IID assumption, we may define the likelihood function as

$$p(D_n | w) := p(y_1, \dots, y_n, x_1, \dots, x_n | w) = \prod_{i=1}^n p(y_i, x_i | w) = \prod_{i=1}^n p(y_i | x_i, w)q(x_i), \quad (2.24)$$

which along with a prior distribution over the weights $\varphi(w)$ gives us, by Bayes' rule, the posterior distribution

$$p(w | D_n) := \frac{p(D_n | w)\varphi(w)}{p(D_n)} = \frac{1}{p(D_n)}\varphi(w) \prod_{i=1}^n p(y_i, x_i | w) = \frac{1}{p(D_n)}\varphi(w) \prod_{i=1}^n p(y_i | x_i, w)q(x_i), \quad (2.25)$$

where

$$p(D_n) = \int_W p(D_n | w)\varphi(w)dw = \int_W \prod_{i=1}^n q(x_i)p(y_i | x_i, w)\varphi(w)dw, \quad (2.26)$$

is the model evidence. We see that in the above the quantities $q(x_i)$ in the numerator and denominator do not depend on w and hence cancel, allowing us to rewrite the posterior as

$$p(w | D_n) = \frac{1}{Z_n}\varphi(w) \prod_{i=1}^n p(y_i | x_i, w), \quad (2.27)$$

where

$$Z_n = \int_W \varphi(w) \prod_{i=1}^n p(y_i | x_i, w) dw, \quad (2.28)$$

is known as the partition function (normalisation constant). Note that the partition function Z_n and the model evidence differ by a factor of $\prod_{i=1}^n q(x_i)$ which does not depend on w . Hence the partition function also gives us an indication of how well the model fits the data (we will see later how this connects to the free energy).

We typically assume a Gaussian model for regression:

$$p(y|x, w) \sim \mathcal{N}(f(x, w), 1_M) \quad (2.29)$$

$$p(y|x, w) = \frac{1}{(2\pi)^{M/2}} \exp\left(-\frac{1}{2}\|y - f(x, w)\|^2\right) \quad (2.30)$$

where $f(x, w)$ is a neural network. We may consider a generalised version of the posterior by introducing a parameter β , known as the inverse temperature, which allows us to adjust the exploration vs. exploitation trade-off when sampling from the posterior (lower β means more exploration, higher means less exploration). Bearing in mind the regression model, this tempered posterior is defined as

$$p^\beta(w|D_n) := \frac{1}{Z_n^\beta} \varphi(w) e^{-n\beta L_n(w)}, \text{ where } Z_n^\beta = \int_W \varphi(w) e^{-n\beta L_n(w)} dw \quad (2.31)$$

2.4.2 Loss Function in Bayesian Statistics

We now examine the connection between the negative log-likelihood that we select as our loss function to minimise in many machine learning tasks and the Kullbeck-Leibler divergence (KL divergence) between our model and the true distribution. We generally select KL divergence as our loss function in the Bayesian setting. As we will see, even in the Bayesian setting we still work with the negative log-likelihood but this does not in fact take us out of the Bayesian setting and into the frequentist setting due to the connection between these loss functions.

The Kullbeck-Leibler divergence between our model $p(y, x | w)$ and the true distribution $q(y, x)$ is defined as

$$K(q || p; w) = \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} q(y, x) \log \frac{q(y, x)}{p(y, x | w)} dx dy, \quad (2.32)$$

and noting that the distribution of inputs $q(x)$ for the experiments in this thesis is known, and writing $K(w)$ instead of $K(q || p; w)$ for each, we may simplify the above

$$K(w) = \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} q(y | x) q(x) \log \frac{q(y | x) q(x)}{p(y | x, w) q(x)} dx dy, \quad (2.33)$$

$$K(w) = \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} q(y | x) q(x) \log \frac{q(y | x)}{p(y | x, w)} dx dy, \quad (2.34)$$

which we see is the expected value of $\log \frac{q(y|x)}{p(y|x,w)}$ w.r.t. the true distribution $q(y, x)$, hence we have that

$$K(w) = \mathbb{E}_{y, x \sim q(y, x)} \left[\log \frac{q(y | x)}{p(y | x, w)} \right]. \quad (2.35)$$

Now applying one of the laws of logarithms and by linearity of the expectation we have that

$$K(w) = \mathbb{E}_{y, x \sim q(y, x)} [\log q(y | x)] - \mathbb{E}_{y, x \sim q(y, x)} [\log p(y | x, w)], \quad (2.36)$$

and bringing minus signs inside the expectation and rearranging we have

$$K(w) = \mathbb{E}_{y, x \sim q(y, x)} [-\log p(y | x, w)] - \mathbb{E}_{y, x \sim q(y, x)} [-\log q(y | x)], \quad (2.37)$$

which we may rewrite as

$$K(w) = L(w) - S, \quad (2.38)$$

where

$$L(w) = \mathbb{E}_{y, x \sim q(y, x)} [-\log p(y | x, w)] = - \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} q(y, x) \log p(y | x, w) dx dy, \quad (2.39)$$

is our negative log-likelihood i.e. the loss function we are seeking to minimise and where

$$S = \mathbb{E}_{y, x \sim q(y, x)} [-\log q(y | x)] = - \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} q(y, x) \log q(y | x) dx dy, \quad (2.40)$$

is the entropy of the true conditional distribution $q(y | x)$. Note again that the distribution of inputs $q(x)$ is known in these examples and so we only need to consider the entropy of the true conditional distribution $q(y | x)$ as opposed to the entropy of the joint distribution $q(y, x)$. As the entropy S does not depend on w we see that minimising the negative log-likelihood is equivalent to minimising the KL-divergence up to a constant.

The set of true parameters that perfectly fit the data is:

$$W_0 := \{w \in W | K(w) = 0\} = \{w \in W | p(y | x, w) = q(y | x)\}, \quad (2.41)$$

which may be empty. Many core results in singular learning theory assume that *realisability* holds, meaning that there is a model in our model class that perfectly realises the true distribution. More formally $\exists_w \in W$ s.t. $K(q || p; w) = 0$. It is possible to extend many of these results to situations where realisability does not hold, so long as the condition of relatively finite variance holds (see Chapter 3, Definition 7 in [41]). In such a situation we would consider the set of optimal model parameters which is defined as

$$W_{\text{opt}} = \{w \in W | L(w) = \min_{w' \in W} L(w')\}. \quad (2.42)$$

2.4.3 Empirical Estimators

Of course, the integrals in $K(w)$, $L(w)$, and S are in many cases intractable to compute so we therefore in practice have the following empirical estimators

$$K_n(w) = \frac{1}{n} \sum_{i=1}^n \log \frac{q(y_i | x_i)}{p(y_i | x_i, w)} = L_n(w) - S_n, \quad (2.43)$$

$$L_n(w) = -\frac{1}{n} \sum_{i=1}^n \log p(y_i | x_i, w), \quad (2.44)$$

$$S_n := -\frac{1}{n} \sum_{i=1}^n \log q(y_i | x_i). \quad (2.45)$$

It is clear that from the above estimators, we have that

$$\mathbb{E}_{y,x \sim q(y,x)} [K_n(w)] = K(w), \quad \mathbb{E}_{y,x \sim q(y,x)} [L_n(w)] = L(w), \quad \mathbb{E}_{y,x \sim q(y,x)} [S_n] = S. \quad (2.46)$$

Furthermore, if it holds that $K(w), L(w), S < \infty$ then as $n \rightarrow \infty$ we have the following almost sure convergences

$$K_n(w) \xrightarrow{\text{a.s.}} K(w), \quad L_n(w) \xrightarrow{\text{a.s.}} L(w), \quad S_n \xrightarrow{\text{a.s.}} S. \quad (2.47)$$

See Chapter 2.3.3 in [4] for the proofs of the above two claims. Crucially, we see that the empirical estimator $L_n(w)$ does not depend on knowledge of the true conditional distribution $q(y | x)$ and hence as $n \rightarrow \infty$ we see that minimising the empirical negative log-likelihood is equivalent to minimising the negative log-likelihood which, as we saw earlier, is equivalent to minimising $K(w)$ up to a constant. This is why we optimise the empirical negative log-likelihood also in the Bayesian setting.

Chapter 3

Singular Learning Theory

The Bayesian information criterion (BIC) [26] is a well-known tool for model comparison/selection

$$\text{BIC} = L_n(w_0) + \frac{D}{2} \log n, \quad (3.1)$$

where

$$L_n(w_0) = \min_{w \in W} L_n(w), \quad (3.2)$$

and D is the number of model parameters. The BIC consists of a term for model (in)accuracy ($L_n(w_0)$) and a term for model complexity $\frac{D}{2} \log n$. Given that the model complexity term is not a function of the location in weight space, but rather a constant term, it is not at all interesting to consider comparing models within the same class for example at different timesteps of the training process.

However, this seemingly uninteresting idea becomes significantly more compelling when we realise that the derivation of the BIC (which is an asymptotic approximation to a quantity known as the free energy, a measure of how well the model fits the data) relies on a Laplace approximation, and that this fails to be applicable whenever the Fisher information matrix contains zero eigenvalues.

This situation arises in almost all of the models that are of interest in machine learning (among them neural networks [44]) meaning that we cannot use the BIC to compare most interesting models in different classes anyway. More importantly, it turns out that a measure of model complexity can be derived that is not necessarily the same for models within a given class. This measure of model complexity can therefore, in principle, be used to examine the increase or decrease of complexity of a given model through training. This brings us to the field of singular learning theory (SLT), a generalisation of statistical learning theory. This section is an attempt to give a short overview of important results and concepts in SLT, please see [4], [5], [6], [7], and [8] for a more comprehensive and intuitive overview.

3.1 Singular Models

We begin with some of the definitions that are given in Chapter 3 of [4].

Definition 3.1.1. Let $W \subseteq \mathbb{R}^D$ and let $p(y|x, w)$ be a given statistical model. Then the Fisher information matrix $\mathcal{I}$ represents the amount of information that a random variable carries about the unknown parameters of p and its elements, $\mathcal{I}(w) = \{\mathcal{I}_{j,k}(w)\}_{j,k=1}^D$ are given by

$$\mathcal{I}_{j,k}(w) = \mathbb{E}_{y,x \sim p(y,x)} \left[\left(\frac{\partial}{\partial w_j} \log p(y|x, w) \right) \left(\frac{\partial}{\partial w_k} \log p(y|x, w) \right) \right], \quad (3.3)$$

$$\mathcal{I}_{j,k}(w) = \int_{\mathbb{R}^N} \int_{\mathbb{R}^M} \left(\frac{\partial}{\partial w_j} \log p(y|x, w) \right) \left(\frac{\partial}{\partial w_k} \log p(y|x, w) \right) p(y|x, w) q(x) dx dy, \quad (3.4)$$

where the derivatives are evaluated at w . We assume $q(x)$ is such that these integrals exist.

Definition 3.1.2. A statistical model $p(y|x, w)$ is identifiable if the map $w \mapsto p(y|x, w)$ is injective for all x, y , and non-identifiable otherwise.

Note that the Fisher information matrix $\mathcal{I}(w)$ is positive semi-definite, meaning that for all $x \in \mathbb{R}^D \setminus \{0\}$ and all $w \in W$, $\mathcal{I}$ satisfies $x^T \mathcal{I}(w) x \geq 0$. It is positive definite when the prior condition holds for the stricter condition of $x^T \mathcal{I}(w) x > 0$, meaning that $\mathcal{I}(w)$ has no zero eigenvalues for any w , and so $\det \mathcal{I}(w) \neq 0$ for all w .

Definition 3.1.3. A statistical model $p(y|x, w)$ is regular if it is both identifiable and has positive definite $I(w)$. It is called strictly singular if it is not regular.

Strictly singular models therefore correspond to models where we have $\det I(w) = 0$ for some $w \in W$. It is a point of understandable confusion in the literature that what is often called a singular model (such as a neural network) is in fact a strictly singular model. The term singular models encompasses both regular and strictly singular models together, capturing the idea that the category of singular models is a generalisation, with regular models as a special case.

This distinction between regular and strictly singular models has “profound consequences for the geometry of $K(w)$, and therefore the learning process” [4]. When a model is regular, within a neighbourhood of a true parameter $w^{(0)} \in W_0$ (note again that W_0 is the set of all parameters $w \in W$ such that $K(w) = 0$), we may approximate $K(w)$ by the quadratic form

$$K(w) \approx \frac{1}{2} (w - w^{(0)})^T I(w^{(0)}) (w - w^{(0)}), \quad (3.5)$$

and then apply standard convex optimisation. However, this approximation is not possible if $I(w^{(0)})$ has zero eigenvalues (we then have a strictly singular model) because the geometry in a neighbourhood of w is then more complex than this simple quadratic form. In the Bayesian context, this statement about the quadratic approximation of the neighbourhood geometry for regular models is equivalent to the statement that the posterior converges in distribution to a normal distribution

$$p(w|D_n) \xrightarrow{d} \mathcal{N} \left(w^{(0)}, \frac{1}{n} I(w^{(0)})^{-1} \right), \quad (3.6)$$

a result known as the Bernstein-von Moses Theorem [34]. Again, if $I(w^{(0)})$ has zero eigenvalues, this asymptotic normality of the posterior does not hold because the inverse of $I(w^{(0)})$ does not exist.

An important related concept here from algebraic geometry is the concept of a singularity. For an analytic function $K : W \mapsto \mathbb{R}$, a singularity w is a critical point of K (so $\nabla K(w) = 0$) for which it additionally holds that $K(w) = 0$. By the non-negativity of $K(w)$ it follows that any $w^{(0)} \in W_0$ is a singularity.

This brings us to the concept of degenerate versus non-degenerate singularities. A singularity w is non-degenerate if we can approximate $K(w)$ in a neighbourhood of w by the quadratic form given in Equation 3.5. Otherwise, it is degenerate. This by itself may not seem to help us but under the assumptions that we have made e.g. that $q(x)$ is a known distribution, that the prior $\varphi(w)$ is a known distribution on a compact space (which W is, as it is finite-dimensional and closed and bounded), and that we are using a Gaussian for the regression model, along

with the assumption of realisability, it follows that the Hessian matrix $H(w) = \mathcal{I}(w)$. We have, by the Morse Lemma [17] that a non-degenerate singularity of the Hessian implies that this singularity is non-degenerate, allowing us to evaluate whether a given singularity is degenerate or not [4]. Note that if realisability does not hold, results in SLT can be extended so long as the assumption of relatively finite variance holds [41].

To summarise, non-degenerate singularities tell us that the geometry around a local minimum has a simple (quadratic) geometric description to which we can apply convex optimisation. For degenerate singularities, the geometry around the singularity is more complex but also requires fewer parameters to describe. This is why we care about the volume of this neighbourhood, the smaller the complexity, i.e. the smaller the number of ‘effective’ parameters, the simpler the model, the better the model will generalise.

3.2 Free Energy and Local Free Energy

Earlier we saw that the partition function Z_n is related to the model evidence and is a measure of how likely the data is under this model. which can be understood in terms of how well the model fits the data vs. how complex the model is. We may reformulate this quantity, as it is described in statistical physics as the minus log marginal likelihood, or the free energy, given by

$$F_n = -\log Z_n, \quad (3.7)$$

or for the general tempered posterior we define it as follows (again assuming that we are using a Gaussian as our regression model):

Definition 3.2.1. Given a dataset D_n , we define the total empirical free energy $F_n^\beta \in \mathbb{R}$ as

$$F_n^\beta = -\log Z_n^\beta = -\log \left(\int_W \varphi(w) e^{-n\beta L_n(w)} dw \right). \quad (3.8)$$

We will refer to F_n^β simply as the free energy for ease.

Of course this quantity is a global indication of how likely the observed data is under this model and could in principle be used to select between different models. However, this quantity can also reveal important local information about the geometry of $K(w)$ and we may therefore compute a local version in order to compare different regions of the parameter space W in terms of how concentrated the posterior is in that region. We restrict ourselves to compact subspaces of W (so in this case closed and bounded by the Heine-Borel Theorem). This leads to the following definition

Definition 3.2.2. For a given β and n , the empirical free energy associated to a compact region $\mathcal{W} \subseteq W$ is

$$F_n^\beta(\mathcal{W}) = -\log Z_n^\beta(\mathcal{W}) = -\log \left(\int_{\mathcal{W}} \varphi(w) e^{-n\beta L_n(w)} dw \right). \quad (3.9)$$

This quantity is known as the local free energy.

3.3 WBIC and RLCT

Of course the integral in Equation 3.2.1 is very often intractable to compute so in practice we cannot evaluate the free energy. Fortunately, there is an alternative. Note that Equation 3.2.1

is dependent on n . Sumio Watanabe was able to show that the free energy has an asymptotic expansion known as the widely applicable Bayesian information criterion (WBIC) which can be estimated. The form gives an approximation to the free energy in terms (restricting to the leading order terms) of the model's fit to the data (as measured by the loss) and the model's complexity, which is expressed in terms of the real log canonical threshold (RLCT), an algebraic invariant from algebraic geometry.

The RLCT can be understood geometrically as the “scaling exponent of the volume of ‘nearly’ true parameters” [5] To understand this, consider the set

$$W_\epsilon = \{w \in W \mid K(w) < \epsilon\}, \quad (3.10)$$

and the associated volume function

$$V(\epsilon) = \int_{W_\epsilon} \phi(w) dw. \quad (3.11)$$

Some further manipulations (see [5] for details) show that we have

$$V(\epsilon) \propto \frac{\epsilon^{\frac{\lambda}{2}}}{\sqrt{c_1, \dots, c_{d'}}}, \quad (3.12)$$

from which we λ appears as the scaling exponent of $\epsilon^{\frac{1}{2}}$. Apply the following ratio of volumes formula for some $a \in (0, 1)$ we obtain an expression for λ as:

$$\lambda = 2 \lim_{\epsilon \rightarrow 0} \frac{\log(V(a\epsilon)/V(\epsilon))}{\log a}, \quad (3.13)$$

where λ is the RLCT. This quantity then gives us a generalisation of the concept of dimensionality that applies to singularities also in strictly singular models (in regular models $\lambda = \frac{D}{2}$). Indeed λ need not be an integer but is a fractional dimension and we have that $\lambda \leq \frac{D}{2}$ and 2λ expresses the number of effective dimensions. Crucially as well, different singularities in W_0 may have different RLCT values [5].

Importantly as well, a smaller RLCT indicates a more complex singularity, suggesting that more singular models may generalize better. This is why SLT offers a possible geometric interpretation for why strictly singular models like deep neural networks seem to depart from the bias-variance trade-off, i.e. they generalise well in spite of having so many parameters.

The following results come from [40], although see Chapter 6 in [39] for more background on the fundamental conditions.

Theorem 3.3.1 (3.8). Consider a (model, truth, prior) triple satisfying the fundamental conditions. Let $\beta = \frac{\beta_0}{\log n}$ for some constant $\beta_0 > 0$, and let $L_n(w_0) = \min_{w \in W} L_n(w)$ (in the realisable case $S_n = L_n(w_0)$). Then there exists a random variable U_n such that

$$\mathbb{E}_w^\beta[nL_n(w)] = nL_n(w_0) + \lambda\beta + U_n \sqrt{\frac{\lambda \log n}{2\beta_0}} + O_p(1), \quad (3.5)$$

where λ is the RLCT of the triple and $\{U_n\}$ is a sequence of random variables which satisfy $\mathbb{E}[U_n] = 0$ and converges in distribution to a Gaussian random variable as $n \rightarrow \infty$.

Theorem 3.3.2. The free energy satisfies

$$F_n = \mathbb{E}_{\beta^*}^w[nL_n(w)] \quad (3.14)$$

at the optimal inverse temperature

$$\beta^* = \frac{1}{\log n} \left(1 + U_n \sqrt{\frac{2}{\lambda \log n}} + o_p \left(\frac{1}{\sqrt{\log n}} \right) \right). \quad (3.15)$$

The quantity $\mathbb{E}_w^{\beta^*}[nL_n(w)]$ is called the Widely Applicable Bayesian Information Criterion (WBIC).

Given the form of Equation 3.14 we see that, as we have access to the numerator of this tempered posterior but not the denominator (as this is typically intractable to compute) we may apply Markov Chain Monte Carlo (MCMC) sampling in order to obtain samples from this posterior that we can then use to compute the expectation. In addition from Equations 3.5 and 3.14 we see that, by disregarding lower order terms, we can obtain an approximation to the free energy in terms of a trade-off between model fit (measured by the loss) and model complexity (measured by the RLCT)

$$F_n(W) \approx nL_n(\omega_0) + \lambda \frac{\log n}{\beta_0}. \quad (3.16)$$

Again note that in the case of regular models we have that the WBIC equals the BIC. We see that in the Bayesian context there is a natural trade-off between model fit and complexity when we optimise the free energy. However, in practice, we are not applying Bayesian learning to train neural networks but rather stochastic gradient descent. Why then do we see something similar occurring in the non-Bayesian setting? This is still an open question, although there is some support for the idea that SGD is somehow implicitly performing Bayesian learning (see for example [23] and [32]), this conjecture is not yet proven to hold.

3.3.1 LLC and wrLLC

Just as we were able to obtain an asymptotic expansion to the free energy through the WBIC, we may do the same with an asymptotic expansion to the local free energy in the form of a local version of the WBIC. This in turn gives us an estimator of a local version of the RLCT, known as the local learning coefficient (LLC) estimator proposed in [20] and given by

$$\hat{\lambda}(w^*) := \frac{E_{w|w^*, \gamma}^{\beta^*}[nL_n(w)] - nL_n(\hat{w}_n^*)}{\log n} \quad (20)$$

where we write, for the expectation of a function $f(w)$ w.r.t. a localised version of the tempered posterior, the following

$$E_{w|w^*, \gamma}^{\beta}[f(w)] := \int f(w) p^{\beta}(w|\mathcal{D}_n, w^*, \gamma) dw. \quad (3.17)$$

This then gives us the ability to quantify the complexity of the geometry around a local minimum during the training of a neural network. In fact, we may extend this result further as described in [37] to compute the complexity of a subset of the parameters around a local minimum $w \in W$ and/or the specialisation of a set of parameters to subsets of the data, leading to the weight-and-data-refined LLC.

Weight- and data-refined LLC (wdrLLC). Consider a data distribution q' (not necessarily

the true distribution q), and let L' and L'_n be the corresponding loss and empirical loss. For a product decomposition $W = U \times V$ corresponding to choosing a set of weights V belonging to a particular component of the model (with U representing the remaining weights/parameters), with the corresponding decomposition of the parameter vector $w^* = (u^*, v^*)$. Consider a neighbourhood B of v^* that is small enough such that $L'(u^*, v) \geq L'(u^*, v^*)$ for all $v \in B$ and define the volume function

$$\text{vol}(\epsilon, w^*, V, q') = \int_{v \in B, |\ell'(u^*, v) - \ell'(u^*, v^*)| < \epsilon} dv. \quad (3.18)$$

The *weight- and data-refined* LLC is then given by

$$\lambda(w^*; V, q') = - \lim_{t \rightarrow 0^+} \log_2 \left[\text{vol}(\frac{1}{2}t, w^*, V, q') / \text{vol}(t, w^*, V, q') \right]. \quad (3.19)$$

Note that in the case that $q' = q$ we refer to Equation 3.19 as the weight-refined LLC (wrLLC) and in the case where $V = W$ we refer to Equation 3.19 as the data-refined LLC (drLLC). Finally, when both $q' = q$ and $V = W$ apply we obtain the LLC.

Chapter 4

Toy Model of Superposition

In this chapter we describe the setup for the experiments of the toy model of superposition setting [13]. The superposition hypothesis suggests that neural networks may represent more features in the input than they have neurons by exploiting the sparsity of input features. In other words, neural networks are trying to ‘noisily’ simulate larger networks with sufficient neurons to represent one feature per neuron. This means some neurons become ‘polysemantic’, i.e. they respond to multiple features [29], allowing the network to simulate this higher capacity ‘disentangled’ model.

Although this ‘polysemanticity’ comes at the cost of interference when more than one of these features represented by a neuron is present in the input, the sparsity of features in the input in general seems to make this a rare occurrence and the non-linearities in the network seem to enable the model to tolerate a certain amount of this interference by filtering it out as noise. This phenomenon is explained and neatly demonstrated in [13].

We train an SAE [27] to take the learned features of the toy model out of superposition, thereby obtaining a representation of this disentangled model to which we can apply weight-refined LLC estimation (Equation 3.19 where V are the respective monosemantic neurons) to analyse the developmental stages of neurons in this disentangled model. We then compare the developmental stages discovered from feature-refined LLC estimation to those found from the LLC of the full disentangled model (applying wrLLC to monosemantic neurons is what we are referring to as feature-refined LLC estimation).

4.1 TMS model

The model we consider for this experiment is the same model used in [13] but with the experimental setup used in [10]. Specifically, we will have a hidden dimensionality of $r = 2$ (so two neurons in the hidden layer) and an input dimensionality of $c = 6$ (so six features can be present in the input). The model is described by the following mapping:

$$f : X \times W \rightarrow \mathbb{R}^c,$$

which is defined as

$$f(x, w) = \text{ReLU}(W^T W x + b), \quad (4.1)$$

where $w = (W, b) \in W \subseteq M_{r,c}(\mathbb{R}) \times \mathbb{R}^c$ and where $x \in X = [0, 1]^c$. Also, as in [10] we assume that the data is obtained from the unknown generating process described by the distribution

$$q(x) = \sum_{i=1}^c \frac{1}{c} \delta_{x \in C_i}, \quad (4.2)$$

where $C_i = \{x \in [0, 1]^c | x_j = 0 \text{ for } j \neq i\}$ denotes the intersection of the i -th coordinate axis with the unit cube $X = [0, 1]^c$. Sampling from $q(x)$ can then be described as

1. Uniformly sample a coordinate i from $\{1, \dots, c\}$,
2. Uniformly sample a length μ from $[0, 1]$,
3. Return μe_i , where e_i is the i -th standard basis vector.

As the above ensures that, for any input $x \in X$, we only have one non-zero element, hence this is the limit of high sparsity in [13]. [10] then propose the following probability model

$$p(x | w) \propto \exp \left(-\frac{1}{2} \|x - f(x, w)\|^2 \right), \quad (4.3)$$

which results in the expected negative log-likelihood function of

$$-\int q(x) \log p(x | w) dx \propto -\int q(x) \log \exp \left(-\frac{1}{2} \|x - f(x, w)\|^2 \right) dx = \frac{1}{2} \int q(x) \|x - f(x, w)\|^2 dx.$$

Discarding terms that are constant w.r.t. w then gives us the following population loss

$$L(w) = \int q(x) \|x - f(x, w)\|^2 dx. \quad (4.4)$$

4.2 Sparse Autoencoder

We will train an SAE after each training epoch of the toy model to learn the features that the toy model has learned after one full pass through the dataset. The SAE has a dictionary size of 48 (so 8 times the number of features) and an L_1 sparsity penalty of 1.0.

4.3 Methods

We train the toy model for 100 epochs on a dataset of size 1000 with a batch size of 20 using the MSE loss as in the original TMS experiments, but in this case without any importance values for the different features. After each full pass through the dataset updating the toy model's parameters we then train a SAE autoencoder (without reinitialising) for up to 1000 epochs to learn the features of the toy model with a stopping criterion of a threshold of 0.9 similarity score between the SAE learned features and the true toy model features. If this threshold fails to be reached, the SAE is trained again for another 1000 epochs (without being reinitialised) and if the threshold is again not met, we continue to the next training epoch.

4.4 Similarity Scores

In the TMS setting, we have access to the ground truth features and so we have a clear way to evaluate the quality of the features that have been learned by the SAE. To make this comparison, we visualise the matrices $W^T W$ for both the toy model and the SAE and examine whether the interference patterns are visually similar. However, this method arguably relies too heavily on visual inspection, so we also compute two quantitative measures of similarity: the mean-max cosine similarity and the similarity score obtained from the Munkres algorithm.

4.4.1 Mean-max cosine similarity (MMCS)

The intuition behind this measure of similarity is to first compute all pairwise cosine similarities, i.e. for each feature in the toy model compute its cosine similarity with all of the learned features in the SAE and then take the max of these and finally take the mean of these max cosine similarities to obtain a measure of similarity. For more details on the reasoning behind this measure of similarity and its use to evaluate the quality of the features learned by an SAE see [31].

4.4.2 Munkres Algorithm

To quantitatively measure how well our SAE features match the true toy model features, we will use the Munkres algorithm. This algorithm finds the optimal one-to-one matching between two sets of features. Also known as the Hungarian algorithm due to the title of the first publication where a similar earlier algorithm was proposed, the version we compute here is described in [24]. Similar to MMCS, we first compute all pairwise similarities and then subtract this from 1 to obtain a cost matrix C (the Hungarian algorithm minimises cost rather than maximising similarity). In the original terms of the assignment problem, the goal is to find the optimal assignment of jobs (toy features) to workers (SAE features) where the optimal assignment is

$$\min_P \text{tr}(PC),$$

where P is a permutation matrix, i.e. a representation of the above assignment by permuting the rows of the cost matrix until we obtain the minimum. In other words, a representation of which SAE features get matched with which toy features. The trace is taken to get the sum of the cost of the matched pairs. The details of how the Munkres algorithm finds this optimal assignment can be found in [24], the important point is that, given how we have defined the cost matrix (in terms of cosine similarities) the above quantity is a number between 0 and 1 that represents the similarity of between the best matching features.

4.4.3 LLC estimation

As the local free energy (Equation 3.9) is, in general, intractable to compute exactly for large models we rely on an estimator (Equation 20) proposed in [20] which computes this expectation w.r.t. to the tempered posterior through stochastic gradient Langevin dynamics [45]. See Section 3.3.1 for more explanation. For smaller models we could compute the expectation through MCMC sampling, but for larger models, it is necessary to use a more efficient alternative such as SGLD. We will apply LLC estimation and the method proposed in [18] to detect the developmental stages of the entire model.

4.4.4 Weight-and-data-refined LLC estimation

Similar to the proposed estimator for the LLC given in [20], we obtain a corresponding estimator for the wrLLC in Equation 3.19, proposed in [37] and defined the expectation $E_{w|w^*, \gamma}^\beta$ by replacing it with the expectation with respect to a Gibbs posterior defined over V by

$$p(v; w^*, q', \beta, \gamma) \propto \exp \left\{ -n\beta \ell'_n(u^*, v) - \frac{\gamma}{2} \|v - v^*\|_2^2 \right\}. \quad (4.5)$$

Recall that V denotes the subset of the weights corresponding to a model component, in our case, one of the (hopefully) monosemantic neurons. See Section 3.3.1 for more explanation.

We will take the learned features of the toy model out of superposition through our use of SAEs and subsequently apply drLLC and wrLLC estimation in order to determine the specialisation of the neurons and analyse their developmental stages. We will then compare these with those of the full model.

4.5 Results

In this section we will present results that aim to address the research questions outlined in Section 1.3,

4.5.1 Training Run for ‘Disentangled’ Model

In Figure 4.1 we see the train and test loss curves for both the toy model and the disentangled model. The fact that the loss curves between the different models are nearly identical gives strong support for the claim that we have obtained an accurate representation of the disentangled model. Thus we obtain an answer for Research Question 1, indeed the loss curves are very similar. See Appendix A for further validation of the extent to which features have been taken out of superposition.

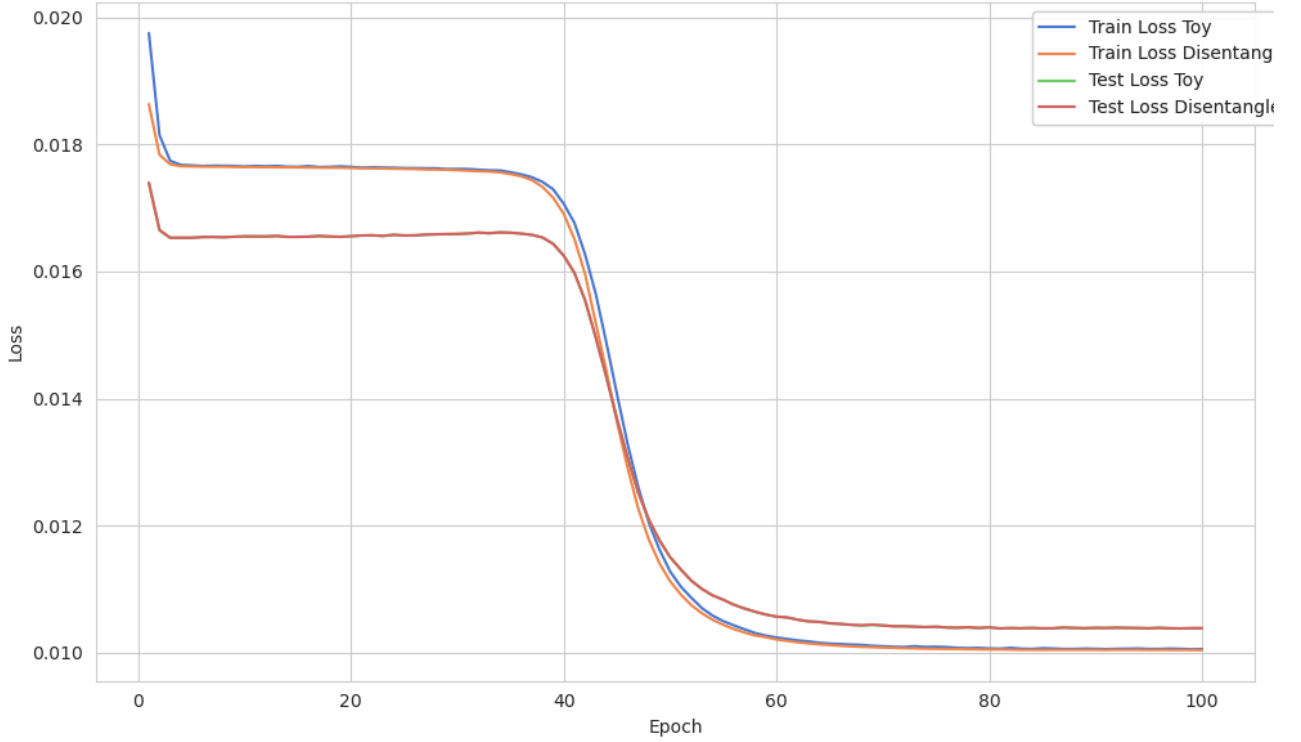


Figure 4.1: Train and test loss for toy and disentangled models

In Figure 4.2 we see the LLC curves for the toy and disentangled models respectively. Here the curves differ noticeably but at least indicate an overall similar trend regarding the steep increase in the LLC around epoch 40. Thus our answer for Research Question 2 is less clear. This discrepancy in the llc curves may not be a failure to take features out of superposition, but rather due to convergence to different estimates at some of the checkpoints during training.

LLC estimation using SGLD requires extensive hyperparameter tuning to ensure convergence and our hyperparameter sweep was likely insufficiently thorough.

To get an impression of whether or not convergence has been achieved we can select a subset of checkpoints from the training run and look at the traces for each of the chains in the SGLD sampling to assess whether or not convergence to an accurate estimate has occurred. These are visible in Figures B.1 and B.2 in Appendix B for the disentangled and full model respectively.

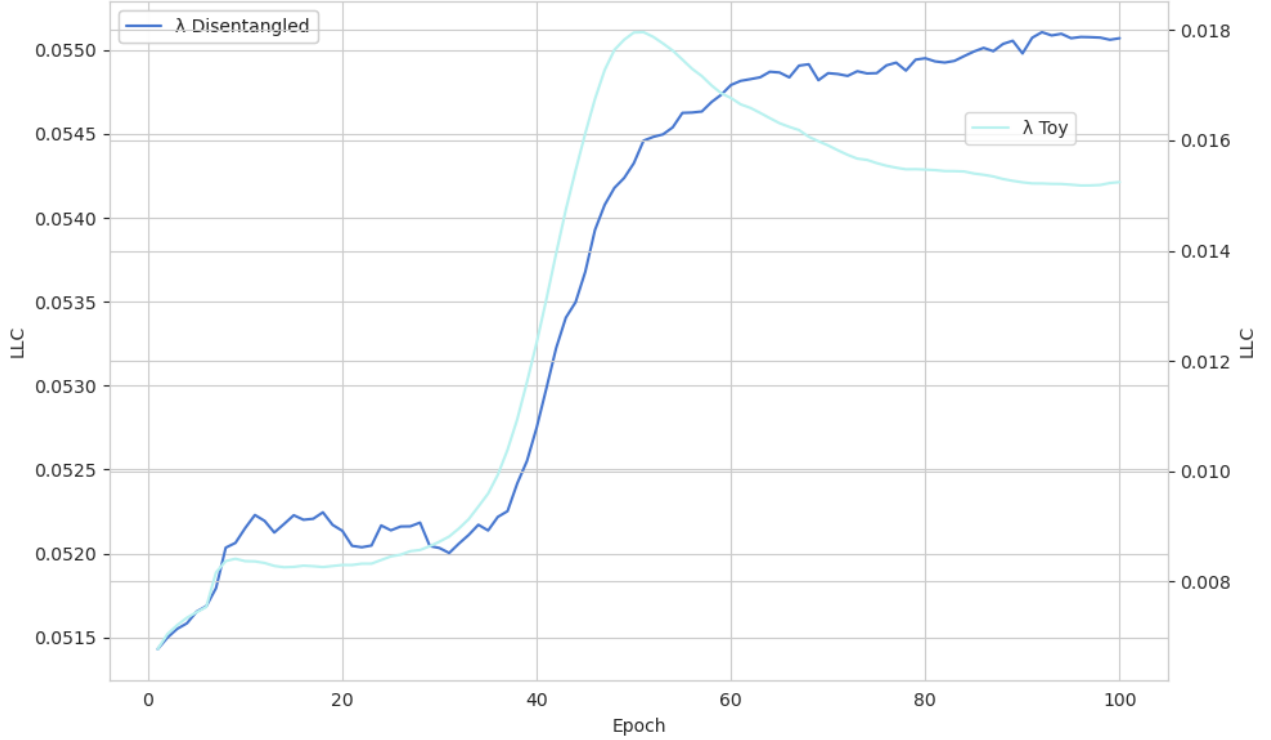


Figure 4.2: LLC for toy and disentangled models

Research Question 1: We see in the above that the curves are very similar, so we may conclude that the answer to this research question is yes.

Research Question 2: Although we see a broadly similar trend above, the curves are not similar enough to conclusively answer yes to this research question.

4.5.2 Specialisation of Features

In the TMS setting, given the structure of the dataset, it is already clear how we can partition the dataset to determine which features are being learned by the monosemantic neurons. Hence there is no need to apply drLLC estimation in this setting. In order to provide a clear answer to Research Question 3 we not only partition the dataset accordingly but also instantiate a pruned model for each feature, i.e. a model where all but one monosemantic neuron has its weights set to zero. We then compute the loss curves for each of these models on each subset of the partitioned data. See Appendix A for more details on how this is carried out. The resulting loss curves for each of the pruned models can be seen in Figures 4.3, 4.4, and 4.5.

In all of these cases, it is quite clear that one neuron performs substantially better than all of the others, except arguably in the case of neuron 5. This neuron does perform better than the others on feature 5 but not to the same extent as the others. This is in line with the observation

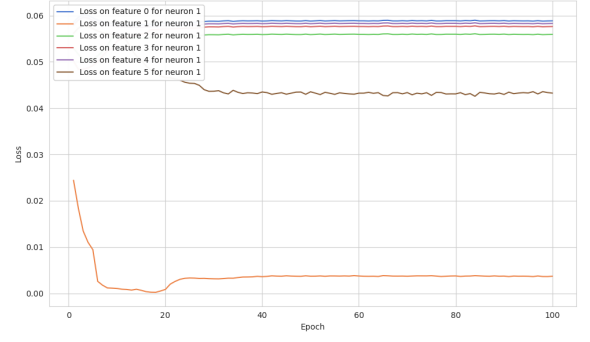
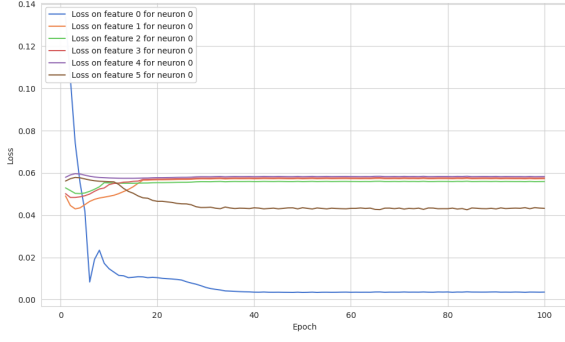


Figure 4.3: Loss per feature for neurons 0 and 1

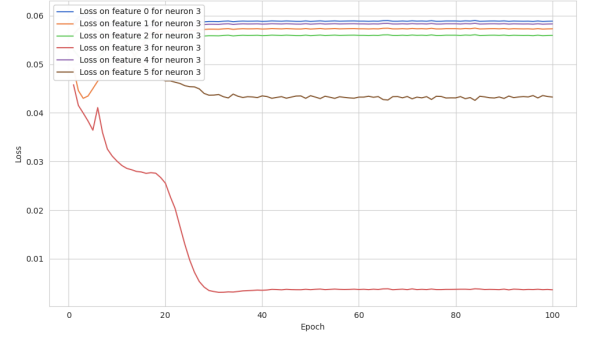
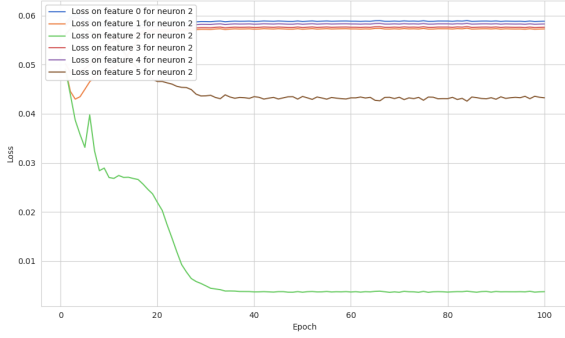


Figure 4.4: Loss per feature for neurons 2 and 3

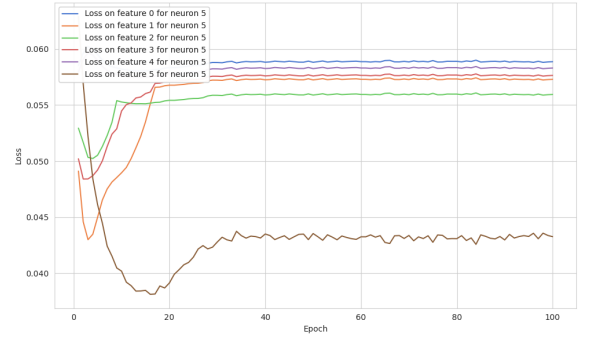
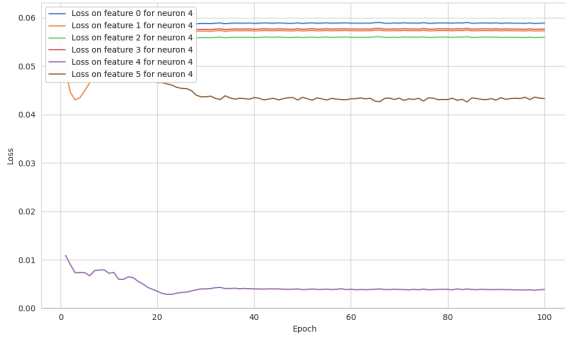


Figure 4.5: Loss per feature for neurons 4 and 5

that feature 5 does not appear to be well-represented in the original toy model. Appendix A for a more detailed discussion. In particular, we can see in Figure A.4 how the disentangled model performs on separate datasets each consisting of a sparse (one nonzero value) vector for each feature.

Research Question 3: We may therefore conclude that, aside from neuron 5, the answer to this research question is clearly yes.

4.5.3 Developmental Stages of Features

In order to provide a clear answer to Research Question 4, we examine the developmental stages that we obtain from applying the approach outlined in [18], i.e. we first apply smoothing the LLC estimates for the training checkpoints (in this case a padded moving average with window size 5) and then fit a Gaussian process regressor to these points and numerically compute the gradient w.r.t. $\log t$ where t is the timestep (checkpoint).

There are two types of transitions between developmental stages we consider as described in [18]:

1. $L(w * \beta) < L(w * \alpha)$ and $\lambda(w * \beta) > \lambda(w * \alpha)$.
(Note that for small n , the loss is large and the LLC is small.)
2. $L(w * \beta) \approx L(w * \alpha)$ and $\lambda(w * \beta) < \lambda(w * \alpha)$.

Detecting transitions between developmental stages of different types is a simple matter of identifying the x-crossings. For transitions between developmental stages of the same type, we must specify a tolerance for the absolute value of the derivative and additionally require that this plateau is a local minimum w.r.t. the absolute value of the first derivative to ensure that multiple adjacent points below the threshold do not get identified as plateaus. In this work we specify a tolerance of 0.01.

We apply the above approach to first compute the developmental stages for the full disentangled model and then for the pruned models, i.e. the neurons. However, before proceeding to this latter task, we first wish to verify that the developmental stages computed for the full disentangled model are meaningful. For more complex models this is a rather extensive task but in the TMS setting we may more reasonably assess this by checking whether the boundaries of stages correspond to meaningful decreases (or increases) in the loss. Thus we can see in Figure 4.6 the loss curve of the full model with the stages indicated. The overall impression is that most meaningful changes in loss do correspond to the boundaries of developmental stages. There are some changes that are unmarked by a boundary and some boundaries that do not seem to correspond to meaningful changes, but this may have more to do with choices for the hyperparameters in computing the stages described above or in the computation of the LLC. See Section 4.6 for a more extensive discussion of these limitations. Setting aside these discrepancies, it seems that overall the stage boundaries are meaningful.

We then proceed to compute the developmental stages of each neuron and visualise these boundaries in relation to the loss curves for the data specialisation of each neuron discussed in Section 4.5.2. Put another way, we take the loss curve for the feature on which a given neuron performs best and compute the developmental stages of this neuron. The results of this can be seen in Figures 4.7, 4.8, and 4.9. The slopes of the LLC curves with the stages indicated for the full model and the neurons can be seen in Figures B.4, B.5, B.6, and B.7 in Appendix B.

Even more so than in the case of the developmental stages of the full model, these results are somewhat ambiguous. For most of the neurons, there is some clear correspondence between stage boundaries and meaningful changes in loss. However, again there are meaningful loss changes that do not have a boundary corresponding to them and boundaries that do not seem to coincide with a meaningful change in loss. Again, as for the full model, see Section 4.6 for a more extensive discussion on these limitations.

Research Question 4: Although we find some support for meaningful boundaries, the above results are not conclusive enough to conclusively answer yes to this research question.

4.5.4 Part-Whole Relationship(s)

In Figure 4.10 we see the LLC estimates of the full model and for each of the neurons. Here we see that each of the neurons has, broadly speaking, the same general shape in its LLC estimate

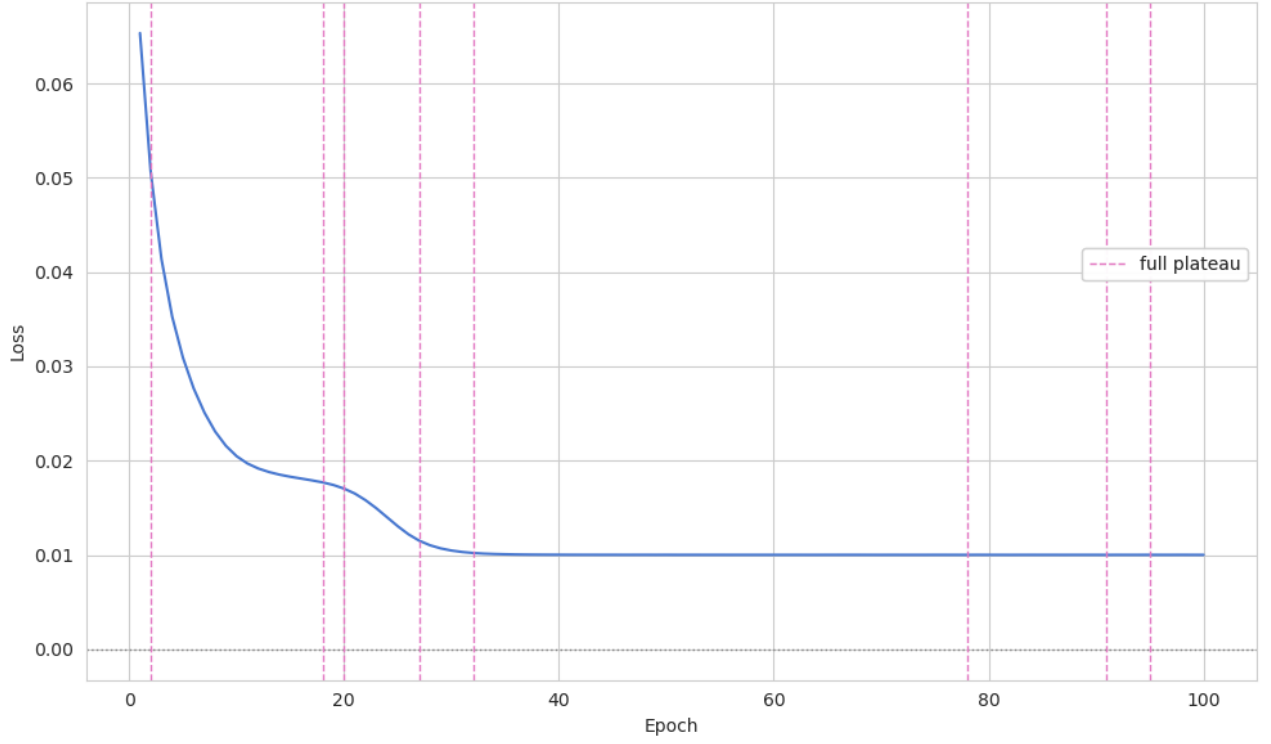


Figure 4.6: Loss of the full disentangled model with developmental stages included.

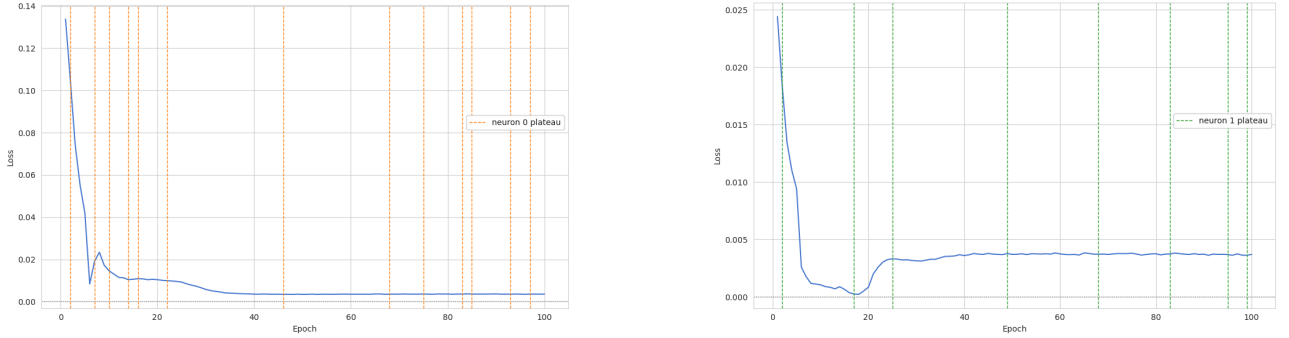


Figure 4.7: Loss curves with developmental stages for neurons 0 and 1.

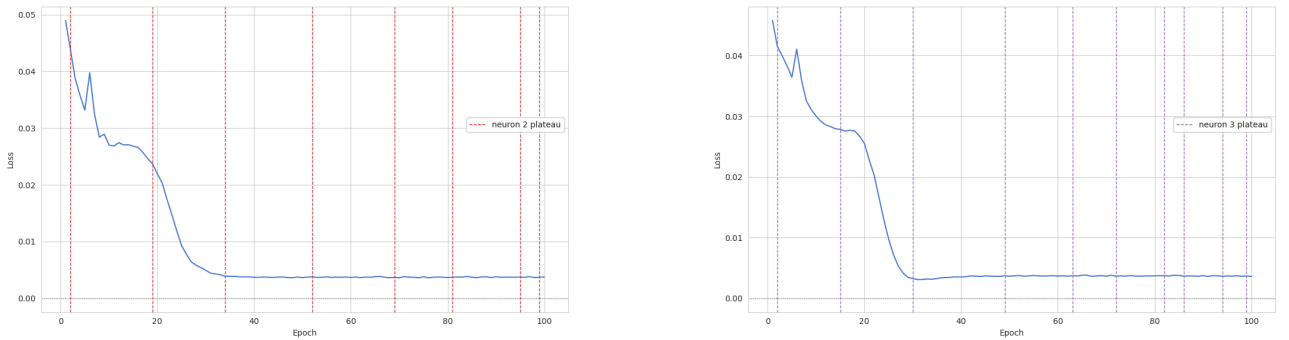


Figure 4.8: Loss curves with developmental stages for neurons 2 and 3.

curve, with the exception of neuron 5. This latter observation can be accounted for by again noting that neuron 5 does not in fact learn to represent a feature by the end of the training process whereas neuron 0 through 4 do. In principle, it seems that the neurons may have different developmental stages. However, it should be borne in mind that LLC estimation is difficult

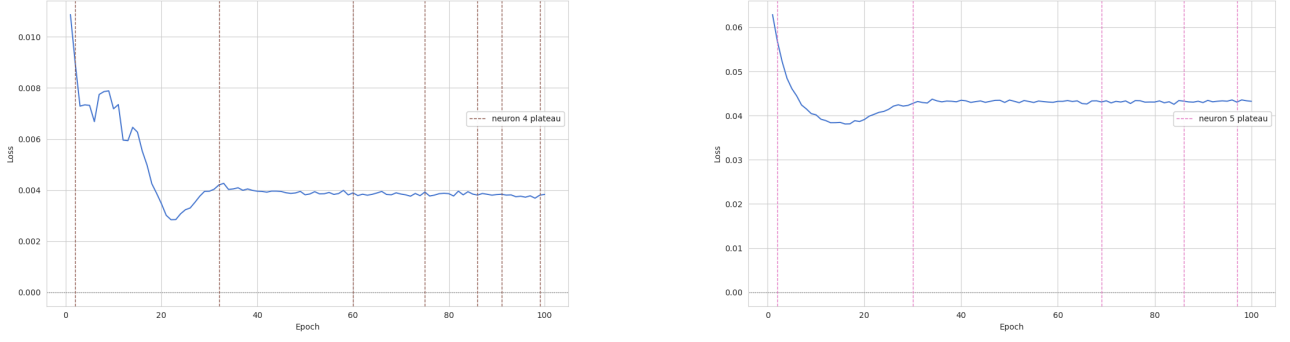


Figure 4.9: Loss curves with developmental stages for neurons 4 and 5.

to get right and SGLD is a more tractable but noisier alternative to MCMC sampling. As a result, it is possible that the differences in the LLC curves for neurons 0 to 4 are explicable by variation in the sampling method used in the estimation process. Even in this case, this still brings up the question of why all of the neurons that learn to represent features would undergo the same developmental trajectory.

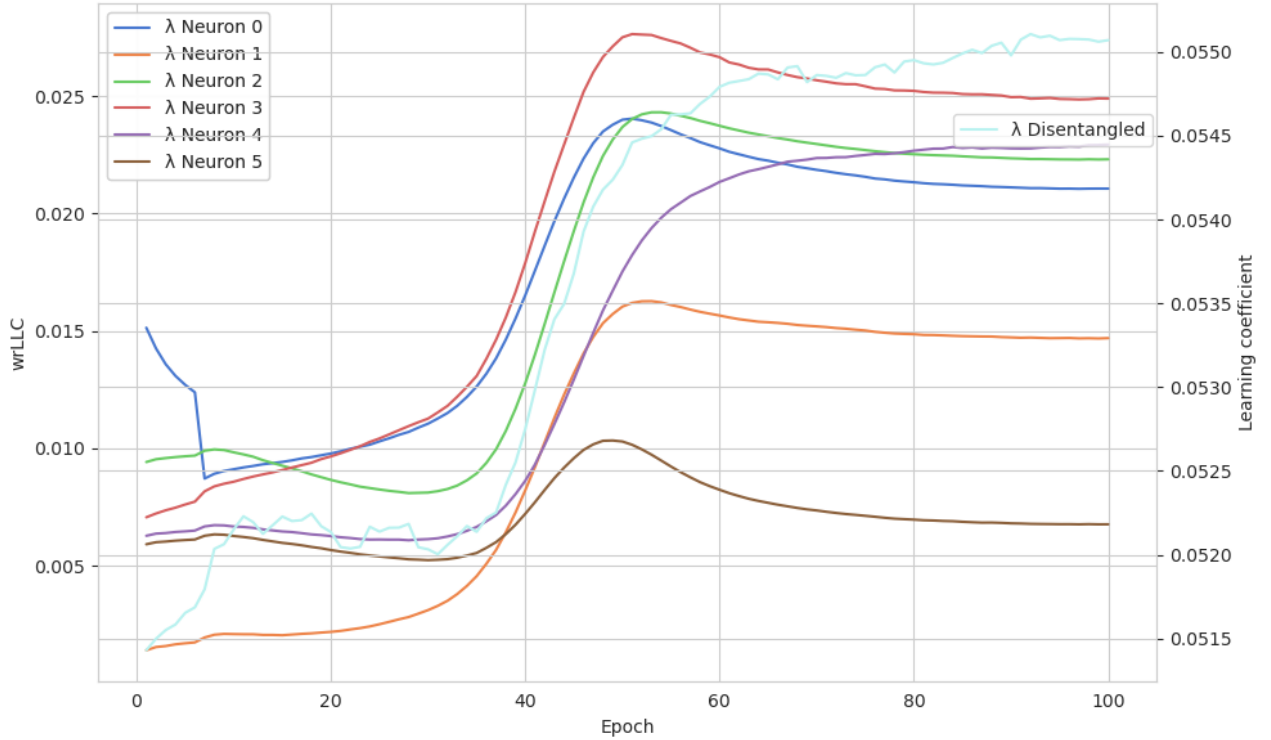


Figure 4.10: Learning coefficient curves for the pruned and full models.

In order to provide an answer to Research Question 5, we compute the elementwise sum of the neuron curves shown in Figure 4.10 and we then visualise this along with the LLC curve of the full disentangled model. The results can be seen in Figure 4.11. Based on the above the answer to this research question is clearly no, it does not seem possible on the basis of this work that the geometry decomposes cleanly into the separate neurons. Again it should be borne in mind that, due to the limitations of LLC estimation and an insufficiently exhaustive hyperparameter search, the above result may not rule out the possibility of this decomposition. It would be instructive to perform a more extensive hyperparameter search and make use of more rigorous SGLD convergence diagnostics in order to assess whether the above is simply an

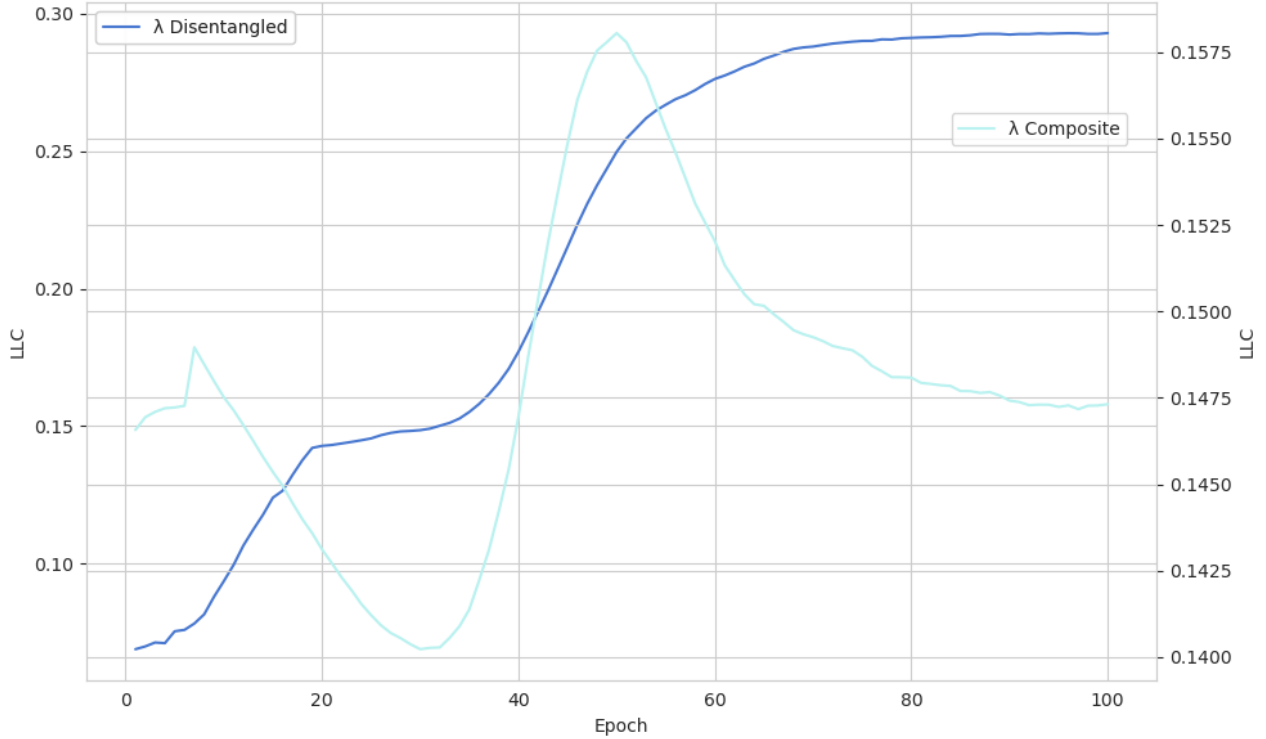


Figure 4.11: Learning coefficient curves for the composite model (sum of the pruned models’ LLC values) and full model.

artifact of this work or whether the proposed hypothesis is simply mistaken. More importantly, from the above, it seems clear that the neurons are not sufficiently monosemantic.

In order to provide an answer to Research Question 6 we again make use of the developmental stages computed for the full model and the neurons, as described in Section 4.5.3, but now we visualise the slope of each neuron and its stages paired together with the slope of the full disentangled model and its stages. The results of this are visible in Figures 4.12, 4.13, and 4.14. It is clear from these results that for each of the boundaries for each of the neurons it is in fact rarely the case that these coincide with one of the boundaries of the full model. Some of the neuron stage boundaries are quite close to boundaries in the full model and so we might try to explain these kinds of discrepancies from the previously mentioned limitations of this approach. However, more importantly, there are many cases of a neuron stage boundary being situated too far away from any of the boundaries of the full model. Therefore this explanation does not hold up to scrutiny and we conclude that the answer to Research Question 6 is no on the basis of this work.

Research Question 5: From the above, we may conclude that the volume of the full model is not decomposable into volume of the distinct neurons.

Research Question 6: From the above we may conclude that the boundaries of the neurons do not coincide with the boundaries of the full model.

4.6 Discussion

We developed a method for feature-refined local learning coefficient estimation and tested it on the toy model of superposition. Our approach involved creating training trajectories for a hypothetical ‘disentangled’ model—where sparse autoencoders remove features from super-

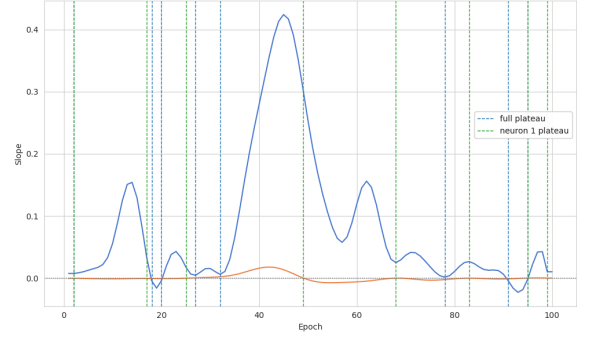
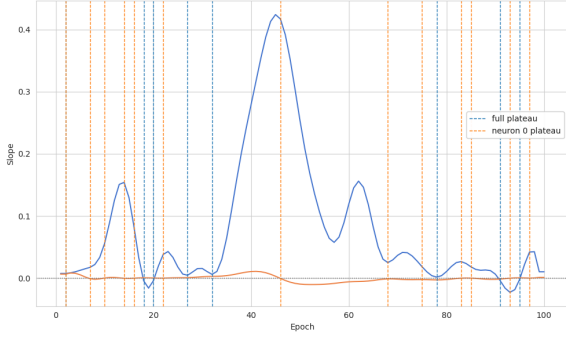


Figure 4.12: Slope of the RLCT for the full model and neurons 0 and 1 along with corresponding stages.

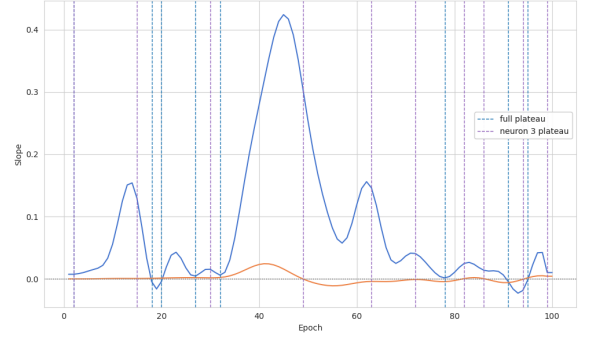
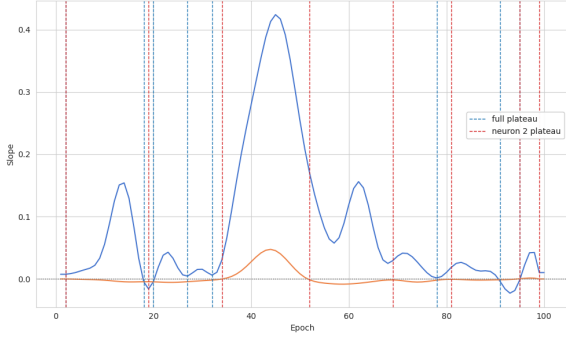


Figure 4.13: Slope of the RLCT for the full model and neurons 2 and 3 along with corresponding stages.

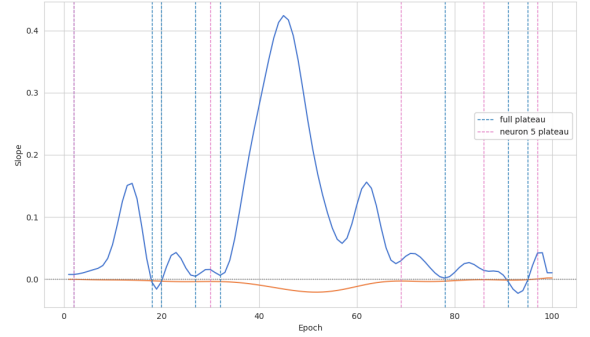
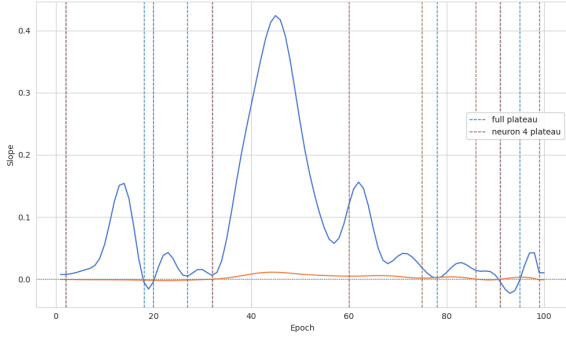


Figure 4.14: Slope of the RLCT for the full model and neurons 4 and 5 along with corresponding stages.

position—then applying LLC estimation to examine how individual monosemantic neurons develop over time. The results offer tentative support for connections between neuron-level and model-level development, but several important limitations affect how we can interpret these findings.

4.6.1 Summary of Main Findings

The experiments yielded mixed results for feature-refined LLC estimation. On the one hand, we showed that SAEs can be trained continuously throughout learning to produce disentangled models whose loss curves closely track the original toy model (Research Question 1). Individual neurons in these disentangled models do specialize on specific features, which our ablation studies confirm (Research Question 3). We can also compute developmental stages for indi-

vidual neurons using weight-refined LLC estimation (Research Question 4), and these stages sometimes correspond to meaningful shifts in the loss curves.

However, our more ambitious objectives received limited support. The LLC curves for original and disentangled models differed substantially, suggesting either incomplete feature disentanglement or problems with the LLC estimation itself (Research Question 2). The full model’s geometry didn’t decompose neatly into individual neurons (Research Question 5), and developmental stage boundaries for individual neurons seldom aligned with those of the complete model (Research Question 6).

4.6.2 Limitation 1: Imperfect Feature Disentanglement

The biggest issue we face is that we cannot verify whether features have actually been taken out of superposition. Our similarity metrics—mean-max cosine similarity and Munkres algorithm scores—suggest reasonable feature recovery, and the loss curves between original and disentangled models track each other closely. But several warning signs point to incomplete disentanglement. This problem gets harder when we do not have access to the ground-truth features.

The differences between LLC curves of the original and disentangled models (Figure 4.2) are noticeable. If we had achieved perfect disentanglement, these complexity measures should agree much more closely, since the models would be functionally equivalent despite following different computational paths.

The interference patterns that characterize the original toy model also persist in our supposedly disentangled representation (Figure A.5). True disentanglement should eliminate these interference effects entirely, yet they remain visible in our results.

A related concern involves consistency across checkpoints. Even though we don’t reinitialize the SAE between training steps, we cannot guarantee that the features extracted at different points correspond to the same underlying representations. The features that neuron 3 responds to at checkpoint 100 might differ from those it responds to at checkpoint 200, even if both show high activation on the same input patterns. Without ground truth feature labels, we lack straightforward ways to assess this consistency. Future work will need better methods for tracking feature identity across time when we don’t have access to the true underlying features.

These disentanglement problems fundamentally limit what we can conclude about feature-level development. If neurons aren’t truly monosemantic, our weight-refined LLC estimates may reflect the complexity of feature mixtures rather than individual features.

4.6.3 Limitation 2: LLC Estimation Challenges

LLC estimation using SGLD presents its own set of reliability concerns. The method depends heavily on hyperparameter choices—learning rate, localisation parameter, noise level, number of chains, number of draws, and burnin steps all require careful tuning. Our analysis shows sensitivity to these choices. The convergence issues we see in toy model traces (Figure B.2)

compared to the disentangled model traces (Figure B.1) illustrate this problem clearly.

The Gelman-Rubin statistics (Figure B.3) add to these concerns. Most of our LLC estimates fail to meet the conventional convergence threshold of $\hat{R} \leq 1.1$. This is especially noticeable for the toy model, where chain mixing looks poor across multiple checkpoints. We computed $\hat{R}$ on WBIC values rather than the underlying samples due to implementation constraints, but this still raises questions about estimate reliability.

SGLD itself represents a computational compromise—more tractable than full MCMC but inherently noisier. The differences we observe between neuron LLC curves might reflect this sampling noise rather than genuine developmental differences. What appear to be meaningful developmental patterns could simply be artifacts of imperfect sampling.

Without more rigorous hyperparameter optimization for each model component and robust convergence diagnostics, we cannot confidently distinguish genuine developmental signals from estimation artifacts. This uncertainty undermines our ability to draw strong conclusions about individual neuron development.

4.6.4 Limitation 3: Developmental Stage Identification

Extracting developmental stages from LLC curves involves several subjective decisions that significantly influence results. We apply a padded moving average with window size 5 before stage identification, but this choice affects whether fluctuations get treated as noise or genuine transitions. Smaller windows would identify more stages (risking false positives), while larger windows might miss real transitions.

Our plateau identification relies on tolerance thresholds for absolute derivatives and local minima detection. Different tolerance values can substantially change both the number of identified stages and their timing. We found that seemingly minor threshold adjustments could produce dramatically different stage patterns.

The derivative computation method introduces additional complexity. We use Gaussian process regression with respect to log time, which brings its own parameters—kernel choice and hyperparameters—that may influence stage identification in subtle ways.

These methodological choices show up clearly in our results. Developmental stage boundaries often fail to correspond with meaningful loss curve changes (Figures 4.7-4.9). Some noticeable changes in loss lack corresponding stage boundaries, while boundaries sometimes appear where no meaningful change occurs.

The accumulation of uncertainties across smoothing, regression, and threshold selection makes it difficult to determine whether identified stages represent real developmental phenomena or methodological artifacts. This problem becomes more pronounced when comparing stages across different models or neurons, since small methodological differences could produce spuriously different patterns.

4.6.5 Implications and Future Directions

While feature-refined LLC estimation remains conceptually appealing, these limitations show that significant methodological improvements are needed before the approach can reliably provide insights into neural network development.

For feature disentanglement, future work should explore more sophisticated SAE training procedures. Better techniques for assessing feature consistency across time will also be crucial.

LLC estimation would benefit from more robust sampling procedures with automated hyperparameter optimization and reliable convergence diagnostics. Alternative complexity measures less sensitive to sampling noise might provide more stable developmental trajectories.

Stage identification needs more principled approaches for choosing smoothing and threshold parameters.

Despite these limitations, our work establishes a foundation for future research in this area. The conceptual framework of feature-refined LLC estimation has value, and the mixed results we observe may reflect the inherent difficulty of this problem rather than fundamental flaws in the approach. The challenges we’ve identified provide a useful foundation that future research might build upon.

Chapter 5

Conclusion

We proposed and demonstrated a method for studying how individual neurons develop in a hypothetical disentangled model where each neuron represents a single feature. Sparse autoencoders trained continuously throughout learning successfully created disentangled models whose loss curves tracked the original toy model. Ablation studies confirmed that individual neurons specialized on specific features, and we found tentative evidence that the developmental stages of the neurons correspond to meaningful changes in the loss curves. This suggests that analyzing individual features could provide new insights into how neural networks develop.

The approach has limitations, however. We cannot reliably verify that features have been truly disentangled, and LLC estimation has convergence problems and high sensitivity to hyperparameter choices. Developmental stage identification also depends on arbitrary methodological decisions that can change the results. In addition, we did not find that the full model’s complexity broke down into the sum of individual neuron complexities, and the developmental stages we identified for individual neurons rarely matched up with those of the complete model. The conceptual framework has potential, but these methodological challenges need to be solved before feature-refined LLC estimation can reliably offer insights on the development of neural networks.

The wider context of this work is to make a contribution towards establishing a falsifiable scientific theory of deep learning. The connection to AI safety is the same connection that is to be found elsewhere in developmental interpretability: if we are to succeed in building highly capable intelligent systems that are also safe for people to use, aligned with human values, and do not pose a risk of serious harm to society, we will need to obtain a deep and precise understanding of how such systems work. Insofar as the current paradigm of deep learning may carry us close to the goal of building systems with high general intelligence, it is crucial that we understand neural networks in a precise and mathematically rigorous way. The work discussed here hopes to make a small contribution to further the development of such a theory by providing an additional means to study the development of structure in such models in a more detailed way.

This does, of course, come with risks. It is very plausible that, equipped with a precise mathematical theory for how neural networks work, this knowledge would be used for harm by various individuals, be that through negligence, incompetence, or outright malice. Nonetheless, in the current climate, there is very little indication that current efforts to build more and more intelligent systems will slow down any time soon. The prospect of having systems that are very generally intelligent and competent, but which we have no deep understanding of how they work seems, if anything, to be a far riskier and dangerous prospect.

Chapter 6

Future Work

In addition to addressing the limitations discussed in Section 4.6, this chapter outlines a proposal for 5 subsequent research projects intended to build upon this work. We hope to apply this approach to progressively more complex toy models in order to try and scale the approach up to real-world models. Each of the models in the subsequent research proposals is progressively more complex, with the exception of Section 6.5 which attempts to investigate the connection between developmental interpretability and the lottery ticket hypothesis [15].

6.1 Multitask Sparse Parity

We propose to apply feature-refined LLC estimation to the multitask sparse parity task. We hope that the structure of this task will make it easier to obtain insights into circuit formation and developmental stages of the model during training. As the data for this task has a clear partition into different subtasks, we will not formally need to apply data-refined LLC estimation.

6.1.1 Background and Motivation

The multitask sparse parity task was proposed in [22] and provides a good testbed for studying circuit formation in neural networks. According to their framework, neural networks learn discrete computational modules (quanta) that correspond to basic skills or algorithms. These quanta are learned in order of decreasing utility, with their usage frequencies following a power law distribution. This hypothesis is known as the Quantization Hypothesis. Quanta and circuits describe the same concept from different perspectives and with different points of emphasis.

6.1.2 Task and Model Setup

The multitask sparse parity dataset consists of n_{tasks} different subtasks, each representing a variant of the sparse parity problem. Given a bit string of length n and a fixed subset $S_i \subset \{1, 2, \dots, n\}$ of size k , the task is to compute the parity (sum mod 2) of the bits at positions in S_i . For example, given a bit-string 010011 of length $n = 6$ and given a subset $S = \{1, 3, 6\}$ (which is a subset of the indices $\{1, 2, 3, 4, 5, 6\}$ of length $k = 3$ in this case), compute the parity of bits 1, 3, and 6 so in this case $(0 + 0 + 1) \bmod 2 = 1$.

Input bit strings have length $n_{\text{tasks}} + n$, where the first n_{tasks} bits are control bits (one-hot encoded to indicate which subtask) and the last n bits are task bits. For a sample where control bit i is active, the correct output is the parity of the task bits at positions S_i .

Following the original experimental setup, we use a one-hidden-layer MLP with ReLU activation. The hidden layer contains 100 neurons to provide sufficient capacity for independent circuit formation and $n_{\text{tasks}} = 100$ and $n = 50$ and $k = 3$. We impose a Zipfian distribution over subtasks during training, with the probability of presenting subtask i being proportional to $i^{-(\alpha+1)}$ where α controls the power law exponent.

We expect the network to form distinct circuits for each subtask, along with a selector circuit that determines which subtask-specific circuit to activate based on the control bits in the input. This creates a natural decomposition where:

1. **Task-specific circuits:** Each subtask requires learning to compute the parity of a specific subset of bits. We anticipate that separate circuits will emerge to handle each of these.
2. **Selector circuit:** The model must learn to interpret the one-hot encoded control bits to determine which subtask is being presented and employ one of the other circuits to solve that subtask.

6.1.3 Feature-Refined LLC Estimation Approach

Feature-refined LLC estimation offers a useful means of studying circuit formation during training. As described in Section 1.3, we reparameterise model complexity from weight space to feature space, measuring the developmental stages of features rather than weights. In this setting, our task is more complicated because the circuits for each subtask will likely comprise multiple features.

Our proposed approach involves the following steps:

1. **Sparse Autoencoder Training:** At each training checkpoint, we apply sparse autoencoders (SAEs) to the hidden layer activations to create a “disentangled” model where features are taken out of superposition. This gives us monosemantic neurons that (we hope) each represent a single interpretable feature.
2. **Circuit Identification (Feature Specialisation):** We identify task-specific circuits and the selector circuit by analyzing which features activate for different input types:
 - Features that activate primarily for inputs from subtask i are candidates for task circuit i
 - Features that activate across all subtasks (particularly in response to control bits) are candidates for the selector circuit
3. **Feature-Refined LLC Estimation:** We apply weight-refined LLC estimation to subsets of parameters corresponding to the task-specific and selector circuits, obtaining developmental trajectories for each.
4. **Developmental Analysis:** We compute developmental stages for the circuits and analyze their relationship to full model developmental stages, seeking to identify when each circuit forms during training and relate this to the stage boundaries of the whole.

6.1.4 Research Questions

As in Section 1.3, we will investigate the following research questions, now in the context of the multitask sparse parity problem (where appropriate with additional explanation):

RQ1: Similar Loss/Accuracy curves Can SAEs trained throughout the learning process create training trajectories for disentangled models that preserve the computational behavior of the original model on multitask sparse parity? (In terms of loss and accuracy curves).

RQ2: Similar LLC curves Similarly, do the training trajectories yield similar LLC curves?

RQ3: Feature Specialisation and Circuit Detection Do collections of features in the disentangled model specialize into distinct circuits for task-specific computation or selector functionality, as predicted by the task structure?

Since we can pre-partition features based on their activation patterns (features activating primarily for subtask i belong to task circuit i , while features activating across all subtasks form the selector circuit), this question examines whether this functional specialisation is clean and consistent. We expect that: (1) task circuits will contain features that activate only for their designated subtask, (2) the selector circuit will contain features that activate in response to control bits across all subtasks, and (3) there will be minimal overlap between circuit memberships. Most importantly, the collection of features we hypothesise to be a circuit can be transparently tested given the task structure. Here it is reasonably clear when we have found a circuit. Note that in this setting, as in the TMS setting, we have no need to apply drLLC estimation.

RQ4: Circuit-Level Developmental Stages Can developmental stages be reliably computed for entire circuits (collections of features) using weight-refined LLC estimation, and do these stages correspond to meaningful changes in the loss and accuracy curves?

Rather than studying individual features, this question focuses on whether we can detect developmental stages for entire functional circuits. We propose computing circuit-level LLC by grouping the weights corresponding to all features within each identified circuit and then applying wrLLC. We expect that developmental stage boundaries will correspond to noticeable improvements in performance on the relevant subtask(s).

RQ5: LLC Decomposition Do the LLC curves of the disentangled model decompose into the LLC curves of the circuits, such that the complexity of the full model equals the sum of complexities of its constituent circuits?

This question tests whether the geometric complexity of the full model can be understood as the sum of the complexities of individual circuits. If circuit formation is truly modular, we hypothesize that $LLC_{full}(t) \approx LLC_{selector}(t) + \sum_{i=1}^{n_{tasks}} LLC_{task_i}(t)$, where each circuit's complexity can be computed independently. Deviations from this relationship would indicate circuit interactions or incomplete disentanglement.

RQ6: Stage Correspondence Between Circuits and Full Model Do the developmental stage boundaries of individual circuits correspond to developmental stage boundaries in the full model, allowing us to decompose model development into circuit formation events?

We will also address the following research question specific to this problem:

RQ7: Circuit Formation Order and Timing Assuming that we can detect the formation of task-specific circuits and the selector circuit, does the order of formation align with the

Quantization Hypothesis predictions?

The Quantization Hypothesis predicts that circuits should form in order of decreasing utility, which in multitask sparse parity corresponds to subtask frequency in the training distribution. We hypothesize that: (1) the selector circuit will form early, as it is essential for all subtasks, and (2) task circuits will form in order of their subtask frequencies (most frequent subtasks first).

6.1.5 Expectations

We hope to demonstrate feature-refined LLC estimation on a well-understood toy problem with known ground-truth circuit structure, to provide insight into circuit formation and testing predictions of the Quantization Hypothesis in a controlled setting.

Some obstacles that may arise include imperfect disentanglement by SAEs, difficulties in LLC estimation for the neurons in the hidden layer, difficulties in mapping discovered features to known computational functions, and ensuring temporal consistency of feature identity across training checkpoints (we do not reinitialise the SAE of course but we may still find a lack of consistency in the learned features across checkpoints that needs to be addressed).

We will behaviorally test whether identified circuits can independently solve their designated subtasks when other components are ablated. We will also apply activation analysis to confirm selective circuit activation patterns, and controlled synthetic data experiments to test circuit functionality. An example of this last approach might be creating inputs that are systematically incorrect for the task to test that the corresponding circuits reliably get this wrong. Or creating correct but carefully designed and partitioned inputs in order to test whether the circuits are actually solving a subtask or something that simply looks like it.

6.2 Modular Arithmetic

We propose to apply the feature-refined local learning coefficient estimation approach to gain deeper insights into the circuit formation process identified by [25]. This represents a natural extension of our method to a more complex algorithmic task where we have some understanding of the underlying mechanisms.

6.2.1 Background and Motivation

The authors [25] demonstrated that transformers learn to perform modular addition by implementing a Fourier multiplication algorithm that maps inputs onto a circle and performs addition through trigonometric identities. Specifically, they identified that the network:

1. Maps inputs a, b to trigonometric functions $\sin(w_k a), \cos(w_k a), \sin(w_k b), \cos(w_k b)$ using sparse key frequencies $w_k = \frac{2k\pi}{P}$
2. Combines these using trigonometric identities to compute $\sin(w_k(a+b))$ and $\cos(w_k(a+b))$
3. Outputs logits proportional to $\cos(w_k(a+b-c))$ for prediction

The training process exhibits three distinct phases: memorization (epochs 0k-1.4k), circuit formation (epochs 1.4k-9.4k), and cleanup (epochs 9.4k-14k), with grokking occurring during the circuit formation phase as the generalizing circuit forms. To detect these transitions, [25] developed several progress measures. The restricted loss measures performance with the model using the Fourier algorithm, while excluded loss measures performance with the model not using it (roughly the generalising and memorising circuits).

6.2.2 Task and Model Setup

We will use the identical experimental setup from [25]: a one-layer transformer with 4 attention heads performing modular addition with prime $P = 113$, trained with 30% of possible input pairs using weight decay $\lambda = 1$. The model uses token embeddings with $d = 128$, learned positional embeddings, attention heads of dimension $d/4 = 32$, and $n = 512$ hidden units in the MLP. We will also use weight decay as this was found to be essential in order for grokking to occur. Without it, the model only memorises the training data.

The task requires predicting $c = (a + b) \bmod P$ given inputs $a, b \in \{0, \dots, P - 1\}$. Based on the mechanistic analysis of [25], we know the network converges to using five key frequencies corresponding to $k \in \{14, 35, 41, 42, 52\}$, with 433 out of 512 MLP neurons implementing degree-2 polynomials of single frequencies.

6.2.3 Feature-Refined LLC Estimation

Unlike the multitask sparse parity case where data naturally partitions into subtasks, modular arithmetic presents the challenge that we do not have an obvious a priori method to partition the data into meaningful subsets. Hence, once we have obtained a training run for the disentangled model, we will apply drLLC estimation once we have identified the appropriate model components. It is clear from previous work why we may consider the attention heads as separate model components [37].

Building on the work described in Chapter 4, we will apply feature-refined LLC estimation to help us identify meaningful model components in the MLP layer and in the embedding layer. However, here too, as in the multitask sparse parity setting, we have too large a number of neurons to apply frLLC estimation to each one. We will therefore prioritise features involved in the structures discovered and analysed in [25]. In particular we will prioritise features involved in:

1. **Frequency Components:** features representing $\cos(w_k a)$, $\sin(w_k a)$, $\cos(w_k b)$, $\sin(w_k b)$ from the embedding layer outputs for each frequency k .
2. **MLP Activations:** features corresponding to some of the 433 neurons identified as implementing degree-2 polynomials of single frequencies, grouped by their associated frequency.
3. **Trigonometric Combinations:** features representing the trigonometric identities $\cos(w_k(a + b))$ and $\sin(w_k(a + b))$ that emerge during circuit formation.

In applying frLLC estimation to such above neurons we may then compute their developmental stages and see if we notice similar boundaries for subsets of the chosen neurons. This would be an indication of neurons performing similar operations. If we can partition our chosen neurons through this approach we may discover model components that form part of a circuit for solving

this task. We will validate the significance of discovered circuits through targeted ablations at transition points. Through this we hope to identify the so-called generalisation circuit and analyse its developmental trajectory. We could then give an answer to the research questions described in the next subsection.

6.2.4 Research Questions

We will investigate the following research questions, adapting our core framework to the modular arithmetic context:

RQ1: Similar Loss/Accuracy Curves Can SAEs trained throughout the learning process create training trajectories for disentangled models that preserve the computational behavior of the original model on modular arithmetic? (In terms of loss and accuracy curves).

RQ2: Similar LLC Curves Do the training trajectories yield LLC curves for the disentangled model that are similar to those of the original model?

RQ3: Feature Specialisation and Circuit Detection Can we identify the emergence of frequency-specific computational circuits through feature-refined LLC estimation, and do these circuits correspond to the algorithmic components identified by [25]?

Since we cannot pre-partition data based on clear subtasks, this question focuses on whether our developmental analysis approach can recover the known circuit structure: (1) embedding circuits that map inputs to trigonometric functions, (2) attention circuits that compute degree-2 polynomials, (3) MLP circuits that implement trigonometric identities, and (4) output circuits that combine frequencies. Most importantly, we will test whether features grouped by our developmental analysis actually implement coherent computational functions.

RQ4: Circuit-Level Developmental Stages Can developmental stages be reliably computed for frequency-specific circuits using weight-refined LLC estimation, and do these stages correspond to the three phases (memorization, circuit formation, cleanup) identified by [25]?

We will compute circuit-level LLC by grouping weights corresponding to all features within each frequency group and applying wrLLC estimation. We hypothesize that developmental stage boundaries will correspond to: (1) the onset of frequency-specific computation, (2) the formation of trigonometric identities, and (3) the sparsification toward key frequencies during cleanup.

RQ5: LLC Decomposition Do the LLC curves of the disentangled model decompose into the LLC curves of the frequency-specific circuits, such that the complexity of the full model equals the sum of complexities of its constituent frequency components?

This tests whether the geometric complexity of the full model can be understood as the sum of the complexities of individual frequency circuits. We hypothesize that $LLC_{full}(t) \approx \sum_{k \in \{14,35,41,42,52\}} LLC_{freq_k}(t)$, where each frequency’s complexity can be computed independently.

RQ6: Stage Correspondence Between Circuits and Full Model Do the developmental stage boundaries of individual frequency circuits correspond to developmental stage boundaries in the full model, allowing us to decompose model development into frequency-specific forma-

tion events?

We will also address the following research questions specific to this setting:

RQ7: Correspondence with [25] Phases Do the phase transitions identified by [25] (memorization, circuit formation, cleanup) correspond to the same phase transitions identified by the developmental approach of [18] when applied to this task? In cases of disagreement, which identified stage boundaries correspond more closely to qualitative changes in model behaviour?

RQ8: Frequency Component Emergence Given that [25] identified that the model learns using five key frequencies ($k \in \{14, 35, 41, 42, 52\}$), can we detect the emergence of each frequency component independently through feature-refined LLC estimation? Do different frequencies emerge at different times during training, and does this correspond to their importance for the overall algorithm?

6.2.5 Expectations

We hope to identify sub-phases within the “circuit formation” stage that [25] identified, potentially showing when different frequency components emerge or when specific trigonometric computations develop. This could give us a deeper insight into the formation of circuits for this task.

More broadly, this work tests whether LLC-based developmental analysis can discover circuits without knowing the data structure in advance. Unlike multitask sparse parity where subtasks are obvious, modular arithmetic requires discovering the circuit structure from scratch. If we could succeed here we might be able to apply this work to more complex tasks.

We expect the same challenges here as described in Section 6.1.5 but also that the circuit identification will be far more challenging.

To validate our findings, we will ablate circuits at detected transition points to test their functionality. We will also track the sparsification of embedding and output matrices in Fourier space, comparing our stage boundaries with the restricted and excluded loss measures from the original work. If features we group together based on developmental analysis actually implement similar computations, this suggests we can use this approach to identify circuits.

6.3 Sorting

Building on the foundation established with modular arithmetic, we propose to apply feature-refined LLC estimation to the task of sequence sorting.

6.3.1 Background and Motivation

We select this problem because we can use Tracr [21] we can compile the sorting program into a suitable decoder-only transformer. The compiled model has a known ground-truth structure and implements a specific sorting algorithm. We therefore have complete knowledge of the model structure that we can use to understand how it solves the task. We then aim to apply our approach of frLLC estimation to analyse the development of structure in this model and obtain deeper insights into circuit formation.

Furthermore, sorting requires explicit sequential reasoning that differs fundamentally from modular arithmetic, making this work an important extension of the previous proposals.

6.3.2 Task and Model Setup

We will train the model on sequences of unique integers, following the `sort_unique` program from Tracr [21]. The task involves input sequences of length n containing unique integers from a fixed vocabulary, with training on 70% of all possible permutations to ensure sufficient data diversity and evaluation on the remaining 30% to assess generalization.

Here we will use a model with two attention-layers with 8 attention heads each, followed by an MLP layer. This choice of architecture is constrained by the result of compiling the `sort_unique` program, i.e. we know that this model can solve the task well. It is also a reasonable extension of the previous work to a slightly more complex model.

6.3.3 Feature-Refined LLC Estimation Approach

Our proposed approach involves the following steps:

1. **Sparse Autoencoder Training:** At each training checkpoint, we apply sparse autoencoders (SAEs) to the hidden layer activations to create a “disentangled” model where features are taken out of superposition, yielding monosemantic neurons that each represent a single interpretable feature.
2. **Algorithmic Component Identification:** We identify sorting-specific circuits by analyzing which features activate for different algorithmic operations:
 - Features that activate for pairwise comparisons between elements.
 - Features that activate for target position computation (selector_width functionality, see Section 4.2 in [21] for further explanation).
 - Features that activate for token rearrangement (swap). operations.
3. **Feature-Refined LLC Estimation:** We apply wrLLC estimation to subsets of parameters corresponding to identified algorithmic components, obtaining developmental trajectories for comparison circuits, position computation circuits, and token movement circuits.
4. **Developmental Analysis:** We compute developmental stages for the algorithmic circuits and analyze their relationship to full model developmental stages, seeking to identify when each circuit forms during training and whether they emerge in the predicted order.

6.3.4 Research Questions

We will investigate the following research questions, now in the context of sorting:

RQ1: Similar Loss/Accuracy curves Can SAEs trained throughout the learning process create training trajectories for disentangled models that preserve the computational behavior of the original model on sorting? (In terms of loss and accuracy curves).

RQ2: Similar LLC curves Similarly, do the training trajectories result in similar LLC curves?

RQ3: Algorithmic Component Detection and Specialisation Do collections of features in the disentangled model specialise into distinct circuits for comparison operations, target position computation, and token rearrangement, as predicted by the known sorting algorithm structure?

Given that sorting requires (1) pairwise comparisons between elements, (2) computation of target positions based on comparison results, and (3) movement of tokens to their sorted positions, this question examines whether functional specialisation emerges cleanly.

RQ4: Circuit-Level Developmental Stages Can developmental stages be reliably computed for entire algorithmic circuits using weight-refined LLC estimation, and do these stages correspond to meaningful changes in sorting capability?

We propose computing circuit-level LLC by grouping weights corresponding to all features within each identified circuit (comparison, position computation, token movement).

RQ5: LLC Decomposition Do the LLC curves of the disentangled model decompose into the LLC curves of the algorithmic circuits, such that the complexity of the full model equals the sum of complexities of its constituent sorting circuits?

If algorithmic component formation is truly modular, we hypothesize that

$$\text{LLC}_{\text{full}}(t) \approx \text{LLC}_{\text{comparison}}(t) + \text{LLC}_{\text{position}}(t) + \text{LLC}_{\text{movement}}(t),$$

where each circuit’s LLC can be computed independently.

RQ6: Stage Correspondence Between Circuits and Full Model Do the developmental stage boundaries of individual algorithmic circuits correspond to developmental stage boundaries in the full model, allowing us to decompose sorting algorithm learning into identifiable stages?

6.3.5 Expectations

Using Tracr gives us a clear advantage: we know exactly how the sorting algorithm should work. We can therefore test whether our approach identifies the three main components (comparison, position computation, token movement) and when they form during training.

Sorting is also more complex than modular arithmetic since it requires multiple sequential steps. If we can detect circuit formation here, it suggests the approach might work on other multi-step reasoning tasks. It is also beneficial to extend the work to a slightly more complex model (2 attention layers rather than 1).

We expect the same challenges here as described in Section 6.1.5. In contrast to the modular arithmetic case, identification of circuits will not present the same challenge given the known structure.

We will ablate the circuits we identify to see if they actually perform their supposed functions. and also compare our discovered circuits directly against Tracr’s compiled structure.

6.4 Greatest Common Divisor

We propose to apply feature-refined LLC estimation to the greatest common divisor (GCD) task.

6.4.1 Background and Motivation

The GCD task was studied in [9], who demonstrated that transformers learn to compute GCD using a sieve algorithm rather than Euclid’s algorithm. (The name sieve presumably comes from the Sieve of Eratosthenes algorithm, though this is not explicitly mentioned in the paper).

Transformers learn to cluster all input pairs with the same GCD and predict them identically with 100% consistency - seemingly learning something like GCD equivalence classes. The model first learns divisibility testing for products of base divisors (which can be tested by examining rightmost digits since divisibility by base factors can be determined from final digits), then “groks” small primes in order through sudden learning events after 100+ epochs of flat loss.

The learned algorithm follows three precise mathematical rules:

1. predictions are deterministic for each GCD
2. correct predictions are products of prime divisors of base B plus small primes learned via grokking
3. $f(k)$ = largest correct prediction that divides k.

This creates a natural decomposition where we expect that circuits emerge for: (1) **base divisor circuits** (testing rightmost digits), (2) **GCD equivalence clustering circuits** (maintaining identical predictions for same GCD), and (3) **prime grokking circuits** that emerge suddenly and completely during discrete learning events.

6.4.2 Task and Model Setup

Following [9], we use a transformer with 4 layers with 8 attention heads per attention layer and 1 MLP layer of dimensionality 512 in the hidden layer after each attention layer. We train the model on base encodings from composite bases (30, 60, 210, 420), chosen for their many small prime factors, enabling easy divisibility testing. We also use a log-uniform distribution of operands and outcomes. We opt for composite bases because this was shown to yield better performance: composite base 420 achieves 38/100 correct GCDs while prime base 11 achieves only 2/100. Similarly, the log-uniform distribution is learned because this was found to balance between memorisation and generalisation. Input pairs up to $M = 10^6$ with 30% data coverage and extended training (1000+ epochs) to capture grokking phenomena.

6.4.3 Feature-Refined LLC Estimation Approach

Our approach involves the following steps:

1. **Sparse Autoencoder Training:** At each training checkpoint, we apply sparse autoencoders (SAEs) to the hidden layer activations to create a “disentangled” model where features are taken out of superposition. This gives us monosemantic neurons that (we hope) each represent a single interpretable feature.

2. **Circuit Identification:** Extract features corresponding to known computational components:
 - **Base divisor circuits** for digit-pattern recognition of prime powers that divide the base.
 - **GCD equivalence clustering circuits** for maintaining identical predictions across pairs with same GCD.
 - **Prime grokking circuits** for small primes that don't divide the base, learned suddenly.
3. **Data-Refined LLC Estimation:** Partition training data by GCD equivalence classes, base divisibility patterns, and prime factorization signatures to detect feature specialisation on specific input types.
4. **Feature-Refined LLC Estimation:** Apply LLC estimation to individual circuits to track their developmental trajectories and identify stage boundaries corresponding to algorithmic learning moments.

6.4.4 Research Questions

We will investigate the following research questions, adapting our earlier work to the GCD context [9]:

RQ1: Similar Loss/Accuracy Curves Can SAEs trained throughout the learning process create training trajectories for disentangled models that preserve the computational behavior of the original model on GCD computation?

This includes preserving the characteristic step-like learning curves where GCDs are learned in sudden batches.

RQ2: Similar LLC Curves Do the training trajectories yield LLC curves for the disentangled model that are similar to those of the original model?

RQ3: Feature Specialisation and Circuit Detection Do collections of features in the disentangled model specialize into distinct circuits corresponding to the known algorithmic components: (1) base divisor circuits for digit-pattern recognition, (2) GCD equivalence clustering circuits maintaining 100% consistency for same-GCD pairs, and (3) prime grokking circuits for non-base primes learned suddenly?

Since [9] identified three mathematical rules governing predictions, this question examines whether circuit specialisation cleanly maps to these algorithmic functions. We can validate discovered circuits against the known sieve algorithm structure and the perfect GCD equivalence class behavior.

RQ4: Circuit-Level Developmental Stages Can developmental stages be reliably computed for entire circuits using weight-refined LLC estimation, and do these stages correspond to meaningful changes in GCD computation capability?

RQ5: LLC Decomposition Do the LLC curves of the disentangled model decompose into the LLC curves of the algorithmic circuits, such that the complexity of the full model equals the

sum of complexities of its constituent circuits? If the sieve algorithm formation is truly modular, we hypothesize that something like $LLC_{full}(t) \approx LLC_{base}(t) + LLC_{clustering}(t) + LLC_{grokking}(t)$ will hold, where each circuit’s complexity can be computed independently. Deviations would indicate circuit interactions or incomplete algorithmic decomposition.

RQ6: Stage Correspondence Between Circuits and Full Model Do the developmental stage boundaries of individual circuits correspond to developmental stage boundaries in the full model, allowing us to decompose GCD algorithm learning into circuit formation events? We expect correspondence between circuit stages and the known phases: base divisor learning, equivalence class consistency, and prime grokking, with the step-like learning curves providing clear validation targets for detected transitions.

6.4.5 Expectations

The GCD task is interesting because [9] identified very specific algorithmic behavior: the model learns to cluster inputs by GCD and follows three precise mathematical rules. This gives us clear targets to look for when applying LLC analysis.

We hope to detect when base divisor learning happens (should be early and fast), when equivalence class clustering emerges, and most importantly, when individual primes get “grokked” - these should show up as sudden developmental stage transitions. The step-like learning curves from the original work give us obvious validation points.

We expect the same challenges here as described in Section 6.1.5 and in addition, as for the modular arithmetic case, that the circuit identification may be quite challenging.

We will ablate features we identify as belonging to each circuit type and see if they actually perform the expected functions. We will also track whether our detected stage boundaries align with the sudden prime learning events that [9] documented. If our approach works, transitions in the LLC curves should correspond to the clear jumps in GCD learning performance.

6.5 Lottery Ticket Hypothesis

Building on the feature-refined local learning coefficient estimation framework developed in Chapter 4, we propose to address a possible contradiction between the lottery ticket hypothesis [15] and findings in developmental interpretability.

6.5.1 The Problem

The lottery ticket hypothesis (LTH) posits that dense, randomly-initialized neural networks contain sparse subnetworks (“winning tickets”) that, when trained from their original initialization, can achieve comparable performance to the full network. The strong form of LTH suggests that these winning tickets can be found simply by pruning the larger network, implying that successful circuits exist from the moment of initialization and require only the removal of interfering parameters to unlock their potential.

However, this view appears to contradict the central claim of developmental interpretability (DevInterp) that computational structure is constructed progressively over the course of training, that pruning alone is insufficient to explain network capabilities. If lottery tickets represent pre-existing circuits, how do we reconcile this with observations of feature development, phase

transitions, and the emergence of increasingly sophisticated computational structures during training?

At the same time, the idea of a simpler, smaller model contained within a larger model exists in Singular Learning Theory (SLT) and compatible with one interpretation of the learning coefficient as measuring “effective dimensionality”, at least for minimally singular models. This suggests that LTH and DevInterp might be reconcilable under certain theoretical frameworks, or that their apparent contradiction clarifies the boundaries when each perspective applies.

Clarifying the relationship between these two ideas, both theoretically and in terms of their empirical predictions would be a valuable contribution to understanding neural network training. It may be that LTH is straightforward to refute in more realistic models or settings with clearer algorithmic structure, where the progressive formation of computational circuits can be more directly observed and measured.

In fact, drLLC and wrLLC are sufficient to investigate this contradiction by tracking the complexity evolution of the specific parameters identified as lottery tickets throughout the entire training process, potentially resolving whether lottery tickets represent genuine pre-existing structure or post-hoc identification of parameters that developed effectively during training. Our approach of frLLC could then be applied to analyse the model in more detail.

6.5.2 Task and Model Setup

As our focus is on transformer models, the first and most crucial task is to replicate the findings in [15] for transformers models. For consistency with other research proposals and because it is the minimal model example featuring both attention and MLP layers, we will again make use of the model used in [25].

We will train the above model on the modular arithmetic task described in Section 6.2. Following the methodology in [15], we will apply iterative magnitude pruning to identify winning tickets at various sparsity levels, then investigate whether these tickets represent genuine pre-existing circuits or parameters that developed during training.

Our approach will apply data-refined LLC (drLLC) estimation to determine the specialisation of the identified lottery tickets, and weight-refined LLC (wrLLC) estimation to track the developmental trajectories of lottery ticket parameters.

Additionally, feature-refined LLC (frLLC) estimation will enable us to examine in some detail the extent to which such lottery circuits form circuits. This will allow for fine-grained analysis of whether specific algorithmic components emerge during training or exist from initialization.

6.5.3 Feature-Refined LLC Estimation Approach

Our approach involves the following steps:

1. **Lottery Ticket Identification:** Apply iterative pruning to identify winning tickets post-hoc, testing generalization across different initializations of the same architecture.
2. **Feature Extraction:** Use sparse autoencoders (SAEs) to identify monosemantic features in MLP layers, extract attention patterns corresponding to specific algorithmic operations, and identify embedding features relevant to input processing.

3. **Data-Refined LLC Estimation:** For each identified lottery ticket, identify the data on which it specialises.
4. **Feature-Refined LLC Estimation:** For each identified lottery ticket, analyse its developmental trajectory.

6.5.4 Research Questions

RQ1: Circuit Formation vs. Selection When lottery ticket parameters are identified post-hoc through iterative pruning, do the complexity trajectories of these parameters during the original training show stable, low complexity throughout training (supporting genuine pre-existing circuits), or dynamic complexity changes with clear developmental phases (supporting circuit formation)?

RQ2: Feature-Level Development Using feature-refined LLC estimation, can we detect whether individual features within lottery ticket subnetworks emerge and develop during training, or exist as stable computational units from initialization?

We hypothesize that if the strong form of LTH is correct, lottery ticket features should maintain stable complexity signatures throughout training, while a Devinterp view would predict complexity transitions corresponding to feature differentiation and specialisation.

RQ3: Temporal Precedence If circuits do form within lottery tickets, do these formation events occur before, during, or after the same circuits form in the full network, and do they follow the same developmental trajectory as identified by standard LLC analysis?

RQ4: Cross-Initialization Consistency Do the same parameters from identical initializations develop into different functional roles depending on training conditions, providing evidence that lottery tickets reflect training-dependent development rather than pre-existing circuits?

This tests role plasticity by training multiple models from identical initializations with different curricula or data orderings and tracking whether lottery ticket parameters implement consistent computational functions.

RQ5: Transient Structure Formation Can we identify lottery tickets that correspond to transient computational strategies that are later abandoned for different solutions during extended training?

The existence of transient intermediate structures would provide strong counterevidence for structure being latent at initialization.

6.5.5 Expectations

We aim to test whether lottery tickets are genuinely special from initialisation or just parameters that happened to develop well during training. If we track the LLC evolution of lottery ticket parameters throughout training, we should see different patterns depending on which of these applies. We also hope to clarify whether perhaps some subset of these lottery tickets formed a basis for what later became a circuit, potentially clarifying that these perspectives are not at odds with each other.

We expect the same challenges here as described in Section 6.1.5 in addition to the fact that

we may even struggle to replicate the LTH finding in transformer models.

We will test if lottery tickets identified in one training run implement the same computational operations when we retrain from the same initialization with different data ordering. If the “lottery ticket” parameters end up doing completely different things, this suggests they’re not special from the start.

Bibliography

- [1] Leonard Bereska and Efstratios Gavves. Mechanistic interpretability for ai safety—a review. *arXiv preprint arXiv:2404.14082*, 2024. URL: <https://arxiv.org/abs/2404.14082>.
- [2] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [3] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023. URL: <https://arxiv.org/abs/2303.12712>.
- [4] Liam Carroll. *Phase transitions in neural networks*. PhD thesis, MSc Thesis at the University of Melbourne, 2021.
- [5] Liam Carroll. Dslt 1. the RLCT measures the effective dimension of neural networks, 2023. URL: <https://www.lesswrong.com/s/czrXjvCLsqGepybHC/p/4eZtmwaqhAgdJQDEg>.
- [6] Liam Carroll. Dslt 2. why neural networks obey Occam’s razor, 2023. URL: https://www.lesswrong.com/posts/CZHwwDd7t9aYra5HN/dslt-2-why-neural-networks-obey-occam-s-razor#Intuitive_Examples_to_Interpret_the_WBIC.
- [7] Liam Carroll. Dslt 3. neural networks are singular, 2023. URL: <https://www.lesswrong.com/posts/tZwaGp5wMQqKh3krz/dslt-3-neural-networks-are-singular>.
- [8] Liam Carroll. Dslt 4. phase transitions in neural networks, 2023. URL: <https://www.lesswrong.com/posts/aKBAYN5LpaQMrPqMj/dslt-4-phase-transitions-in-neural-networks>.
- [9] Francois Charton. Learning the greatest common divisor: explaining transformer predictions. In *The Twelfth International Conference on Learning Representations*, 2024.
- [10] Zhongtian Chen, Edmund Lau, Jake Mendel, Susan Wei, and Daniel Mufet. Dynamical versus bayesian phase transitions in a toy model of superposition. *arXiv preprint arXiv:2310.06301*, 2023. URL: <https://arxiv.org/abs/2310.06301>.
- [11] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023. URL: <https://arxiv.org/abs/2309.08600>.

- [12] Nelson Elhage, Tristan Hume, Catherine Olsson, Neel Nanda, Tom Henighan, Scott Johnston, Sheer ElShowk, Nicholas Joseph, Nova DasSarma, Ben Mann, Danny Hernandez, Amanda Askell, Kamal Ndousse, Andy Jones, Dawn Drain, Anna Chen, Yuntao Bai, Deep Ganguli, Liane Lovitt, Zac Hatfield-Dodds, Jackson Kernion, Tom Conerly, Shauna Kravec, Stanislav Fort, Saurav Kadavath, Josh Jacobson, Eli Tran-Johnson, Jared Kaplan, Jack Clark, Tom Brown, Sam McCandlish, Dario Amodei, and Christopher Olah. Softmax linear units. *Transformer Circuits Thread*, 2022. <https://transformer-circuits.pub/2022/solu/index.html>.
- [13] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022. URL: <https://arxiv.org/abs/2209.10652>.
- [14] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 1(1):12, 2021.
- [15] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018. URL: <https://arxiv.org/abs/1803.03635>.
- [16] Andrew Gelman and Donald B Rubin. Inference from iterative simulation using multiple sequences. *Statistical science*, 7(4):457–472, 1992.
- [17] Robert Gilmore. *Catastrophe theory for scientists and engineers*. Courier Corporation, 1993.
- [18] Jesse Hoogland, George Wang, Matthew Farrugia-Roberts, Liam Carroll, Susan Wei, and Daniel Murfet. The developmental landscape of in-context learning. *arXiv preprint arXiv:2402.02364*, 2024. URL: <https://arxiv.org/abs/2402.02364>.
- [19] Adam S Jermyn, Nicholas Schiefer, and Evan Hubinger. Engineering monosemanticity in toy models. *arXiv preprint arXiv:2211.09169*, 2022. URL: <https://arxiv.org/abs/2211.09169>.
- [20] Edmund Lau, Daniel Murfet, and Susan Wei. Quantifying degeneracy in singular models via the learning coefficient. *arXiv preprint arXiv:2308.12108*, 2023. URL: <https://arxiv.org/abs/2308.12108>.
- [21] David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Tom McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. *Advances in Neural Information Processing Systems*, 36, 2024.
- [22] Eric Michaud, Ziming Liu, Uzey Girit, and Max Tegmark. The quantization model of neural scaling. *Advances in Neural Information Processing Systems*, 36, 2024.
- [23] Chris Mingard, Guillermo Valle-Pérez, Joar Skalse, and Ard A Louis. Is sgd a bayesian sampler? well, almost. *Journal of Machine Learning Research*, 22(79):1–64, 2021.
- [24] James Munkres. Algorithms for the assignment and transportation problems. *Society for Industrial and Applied Mathematics*, 15:196–210, 1962.

- [25] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *arXiv preprint arXiv:2301.05217*, 2023. URL: <https://arxiv.org/abs/2301.05217>.
- [26] Andrew A Neath and Joseph E Cavanaugh. The bayesian information criterion: background, derivation, and applications. *Wiley Interdisciplinary Reviews: Computational Statistics*, 4(2):199–203, 2012.
- [27] Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.
- [28] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 5(3):e00024–001, 2020.
- [29] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. Feature visualization. *Distill*, 2(11):e7, 2017.
- [30] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022. URL: <https://arxiv.org/abs/2209.11895>.
- [31] Lee Sharkey, Dan Braun, and Beren. Taking features out of superposition with sparse autoencoders, 2022. URL: https://www.lesswrong.com/posts/z6QQJbtpkEAX3Aojj/interim-research-report-taking-features-out-of-superposition#We_need_more_learned_features_than_ground_truth_features.
- [32] Mandt Stephan, Matthew D Hoffman, David M Blei, et al. Stochastic gradient descent as approximate bayesian inference. *Journal of Machine Learning Research*, 18(134):1–35, 2017.
- [33] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL: <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [34] Aad W Van der Vaart. *Asymptotic statistics*, volume 3. Cambridge university press, 2000.
- [35] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [36] Conrad Hal Waddington. *The strategy of the genes*. Routledge, 2014.
- [37] George Wang, Jesse Hoogland, Stan van Wingerden, Zach Furman, and Daniel Murfet. Differentiation and specialization of attention heads via the refined local learning coefficient. *arXiv preprint arXiv:2410.02984*, 2024. URL: <https://arxiv.org/abs/2410.02984>.
- [38] Sumio Watanabe. Almost all learning machines are singular. In *2007 IEEE Symposium on Foundations of Computational Intelligence*, pages 383–388. IEEE, 2007.
- [39] Sumio Watanabe. *Algebraic geometry and statistical learning theory*, volume 25. Cambridge university press, 2009.

- [40] Sumio Watanabe. A widely applicable bayesian information criterion. *The Journal of Machine Learning Research*, 14(1):867–897, 2013.
- [41] Sumio Watanabe. *Mathematical theory of Bayesian statistics*. chapman and hall/cRc, 2018.
- [42] Sumio Watanabe. Recent advances in algebraic geometry and bayesian statistics. *Information Geometry*, 7(Suppl 1):187–209, 2024.
- [43] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022. URL: <https://arxiv.org/abs/2206.07682>.
- [44] Susan Wei, Daniel Murfet, Mingming Gong, Hui Li, Jesse Gell-Redman, and Thomas Quella. Deep learning is singular, and that’s good. *IEEE Transactions on Neural Networks and Learning Systems*, 34(12):10473–10486, 2022.
- [45] Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 681–688. Citeseer, 2011.

Appendix A

Validation TMS

Aside from the main indicators of the loss (or accuracy curves) for a task being similar, for the TMS setting we can make use of other metrics for assessing whether the features have been taken out of superposition.

We can also see in Figures A.2 and A.3 the similarity scores and mean-max cosine similarity scores between the toy features and SAE features respectively. We also see the matrix products $W^T W$ in Figure A.1 of both the SAE and toy model showing the learned feature representations.

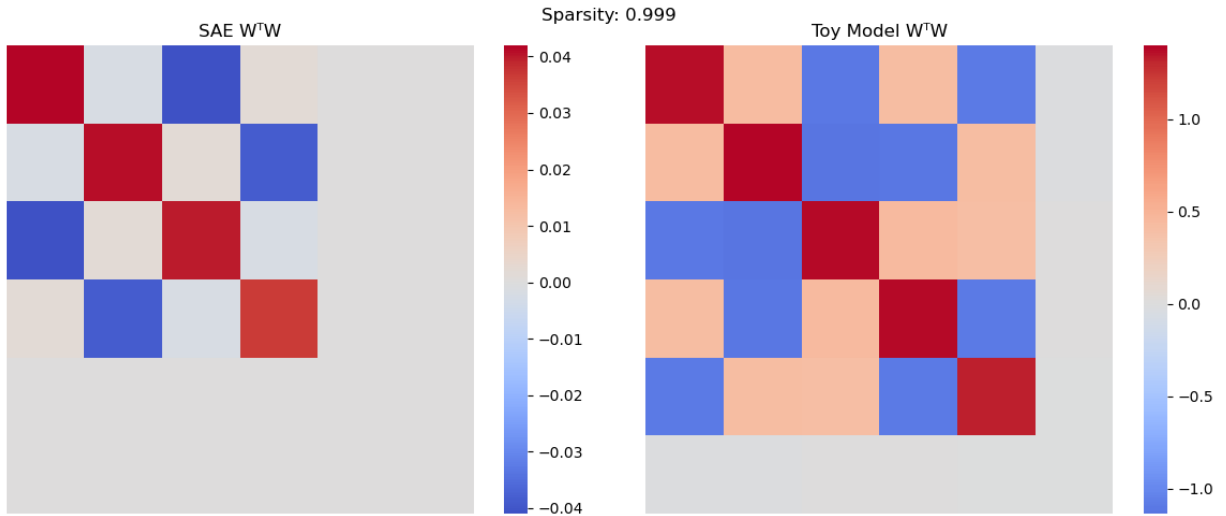


Figure A.1: Comparison of $W^T W$ matrices for the SAE (left) and toy model (right) at sparsity 0.999. The toy model shows more structured feature organization compared to the SAE.

As a final check we can perform a simple ablation analysis to verify that indeed the neurons that we have identified seem to indeed be representing the feature we think they represent. For this simple ablation we construct a separate dataset for each of the features, i.e. c datasets each containing only instances with only one non-zero value at the index of a particular feature. Each dataset consists of 100,000 instances. We then, for each of the neurons in our disentangled model, prune (i.e. set to zero) all of the other neurons and test the model on each of the c datasets. The results of this process are shown in Table A.1 (in terms of MSE error on each of the datasets). It is clear from the above that neurons 0 to 4 score significantly lower loss on features 0 to 4 respectively, and that neuron 5 does not score significantly lower loss for any of the c features. (Note it may be the case that all neurons try to represent feature 5 to an insignificant extent as the loss is lower for that feature (aside from the feature that it properly

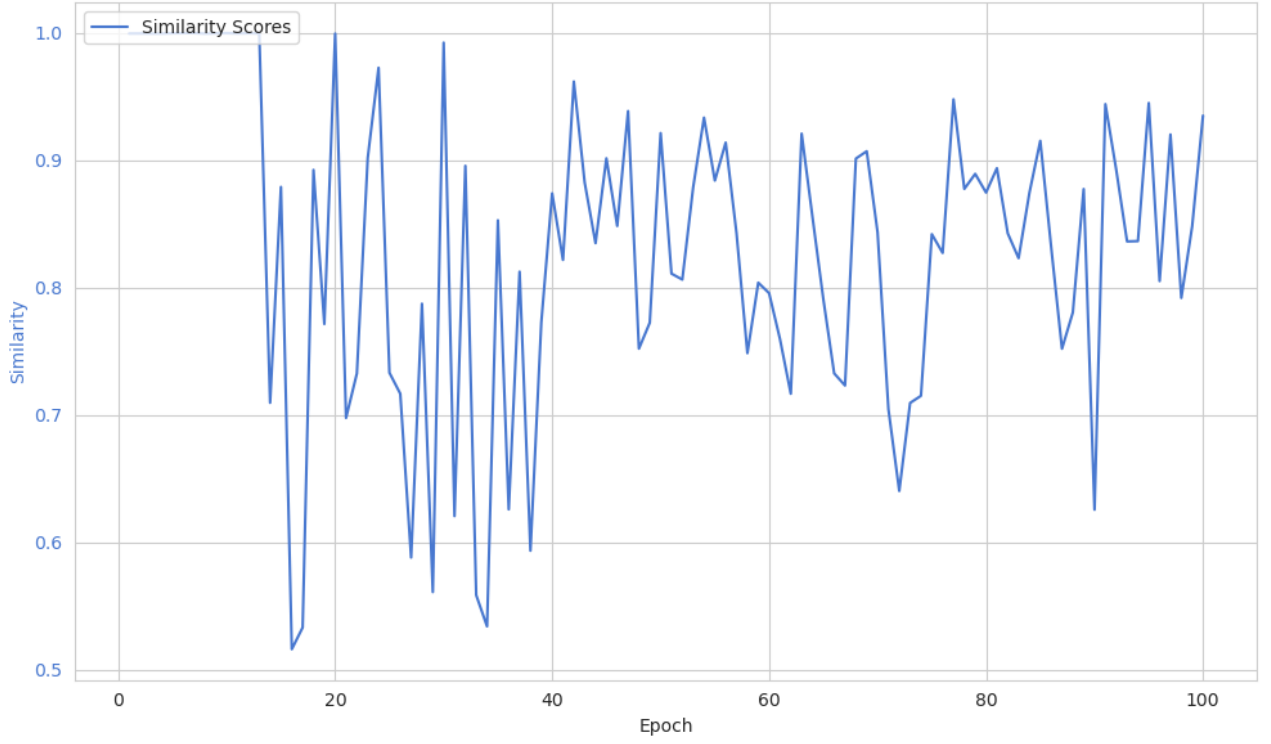


Figure A.2: Similarity scores between SAE and toy model

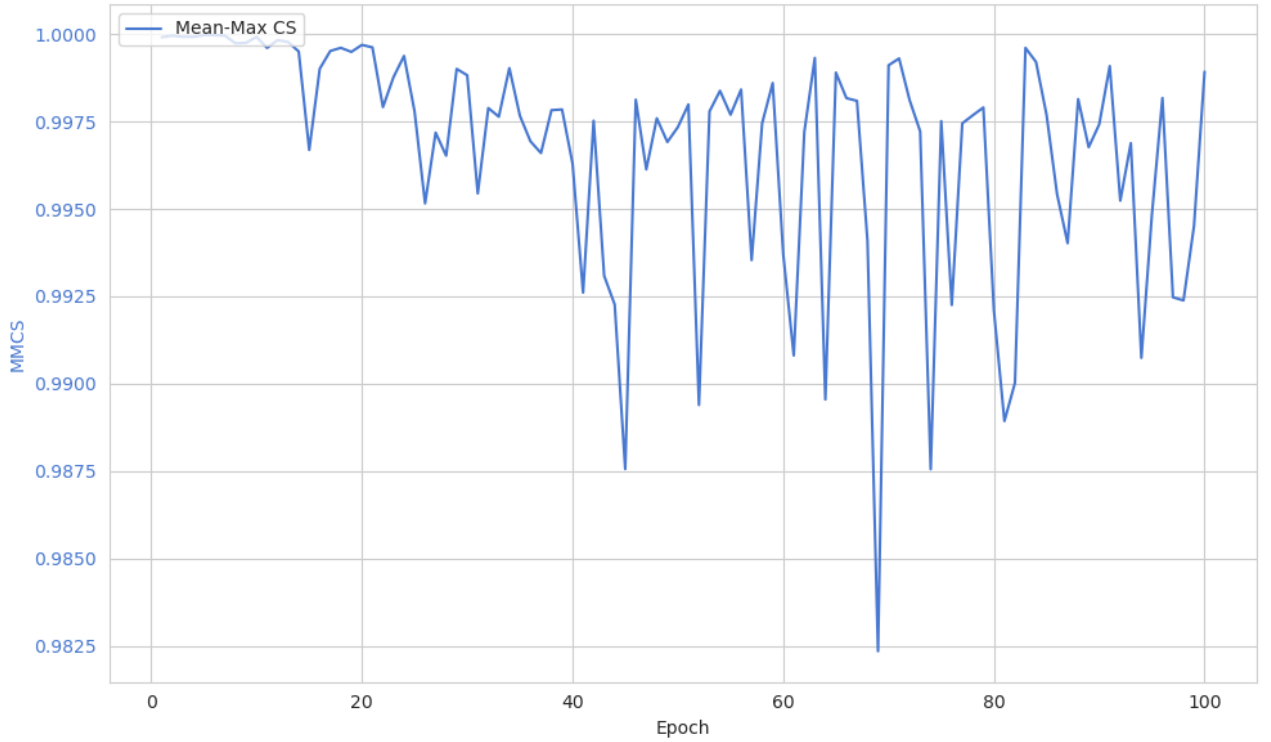


Figure A.3: Mean-max cosine similarity between SAE and toy model

represented) across all neurons but we did not investigate this any further as the difference seemed insignificant).

We also check that the disentangled model's trajectory during training for each of the sep-

Table A.1: MSE loss per feature

	Feature 0	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5
Neuron 0	0.00330	0.05660	0.05652	0.05647	0.05664	0.04319
Neuron 1	0.05685	0.00396	0.05652	0.05647	0.05664	0.04319
Neuron 2	0.05685	0.05660	0.00430	0.05647	0.05664	0.04319
Neuron 3	0.05685	0.05660	0.05652	0.00389	0.05664	0.04319
Neuron 4	0.05685	0.05660	0.05652	0.05647	0.00339	0.04319
Neuron 5	0.05685	0.05660	0.05652	0.05647	0.05664	0.04318

arate ablation datasets, indicated in Figure A.4 and this is also in line with our expectations: the model begins representing the first 4 features with neurons 0 through to 3 hence we do not see any noticeable decrease in loss there, although it is noteworthy that the loss for these neurons increases slightly around epoch 40 which is also where the loss in the full model decreases quite rapidly. Similarly, neuron 4 shows a very rapid drop in the loss around a similar time, as we would expect under the assumption that this neuron suddenly learns to represent feature 4. This precipitous decrease in the loss of neuron 4 coupled with the slight increase in the loss of neurons 0 to 3 is likely connected to the fact that the toy model is representing these features in superposition and around this time has to ‘unlearn’ or undo part of its representation in order to successfully represent a 5-th feature. Finally we note that the loss for neuron 5 shows no significant decrease. This is also in line with our expectations as this last neuron does not appear to be representing any feature (again the very slight drop in loss may be due to this neuron representing the last feature in a very limited way but we did not confirm whether this is of any statistical significance). Finally, we check whether the interference pattern present in

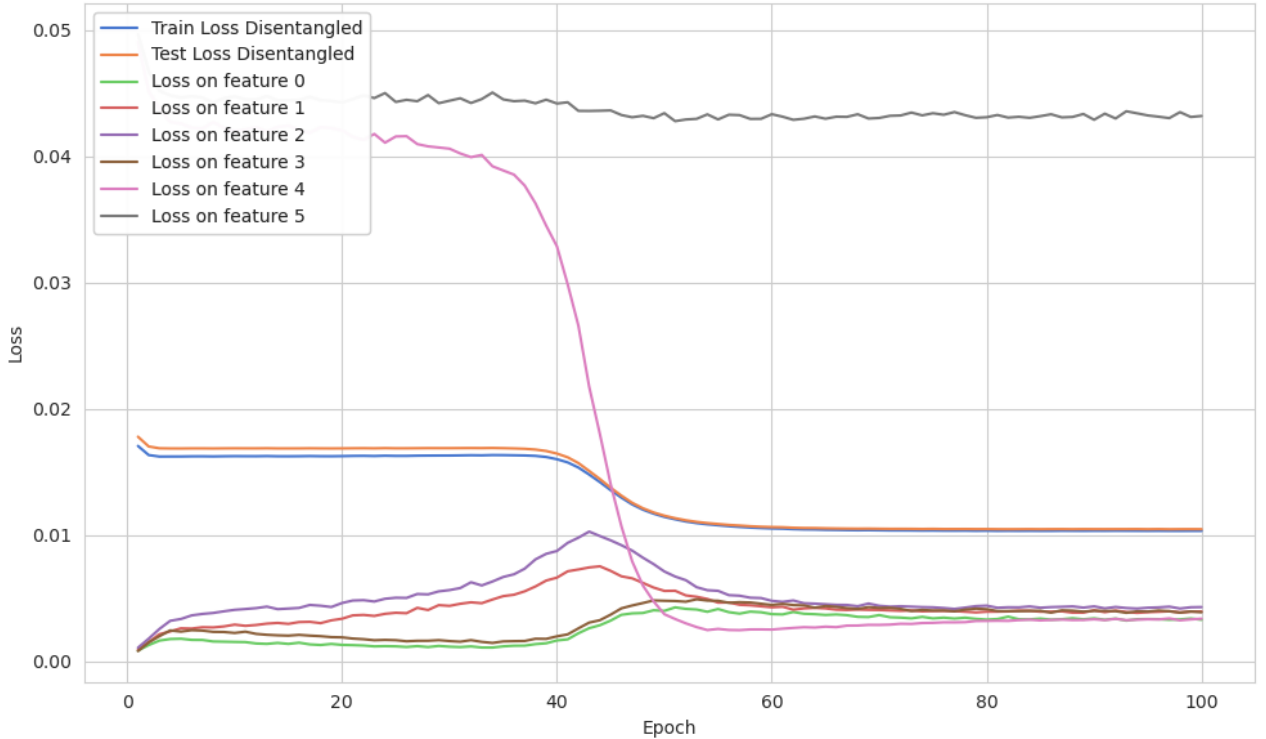


Figure A.4: Losses per feature during training

the toy model is indeed still present in the disentangled model and we find that indeed it is as can be seen in Figure A.5 where the curves are identical. Here we have combined the ablation datasets into one so that each observation contains non-zero values for all features, hence the

situation of zero sparsity.

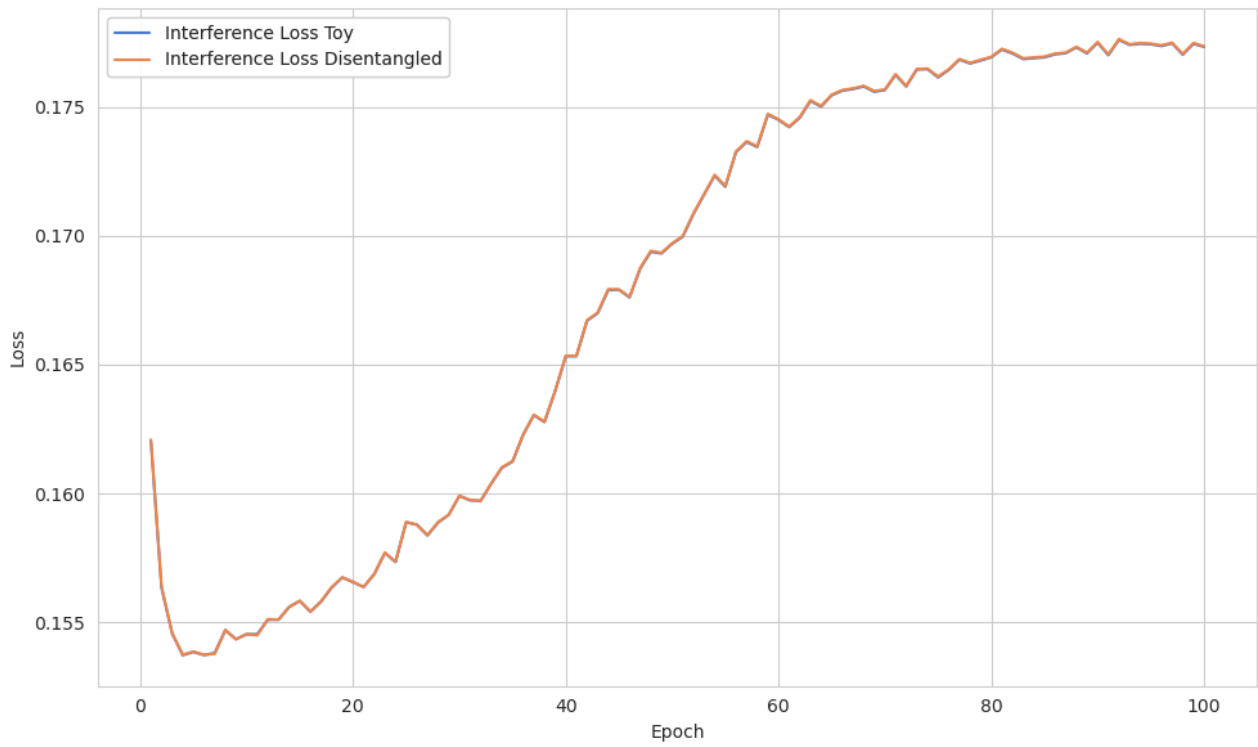


Figure A.5: Losses with dataset containing all features at once throughout trained model's trajectory.

Appendix B

Validation of LLC convergence

Figures B.1 and B.2 show the WBIC trace plots for the different chains in the SGLD sampling for the disentangled and toy models respectively. For the disentangled model we see that there is convergence and mixing of the chains suggesting that the chains are exploring similar regions in parameter space while for the toy model we see that there are not clear signs of convergence and mixing of the chains and this may account for the noticeable difference between the LLC curves.

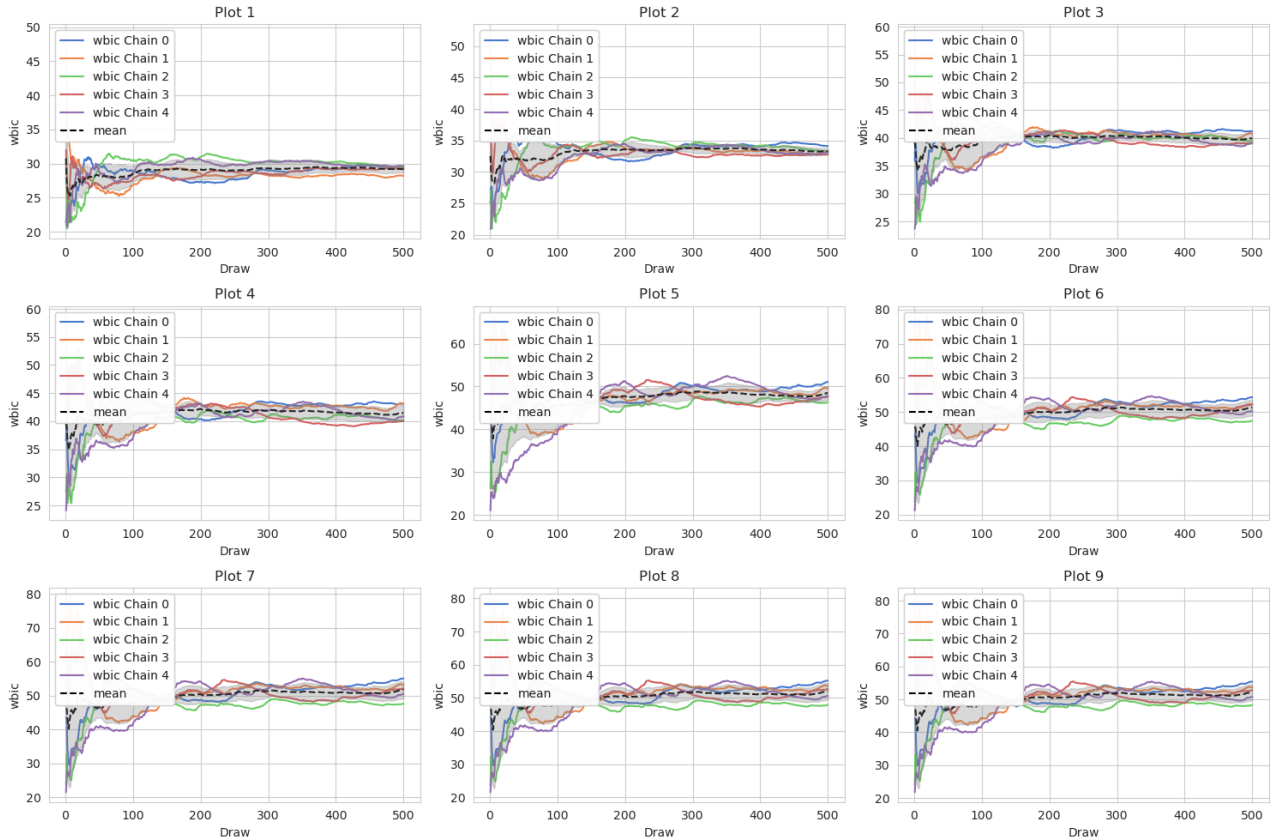


Figure B.1: WBIC trace plots for several snapshots of the total checkpoints for the disentangled model

To further evaluate the presence or absence of convergence we compute the Gelman-Rubin statistic outlined in [16] on the WBIC trace of the chains for each of the LLC estimations at each of the checkpoints. This offers some insight into the convergence of the LLC estimates across training and can be seen in Figure B.3. According to this statistic, a value of less than or equal to 1.1 indicates convergence and so it would seem by this that the estimates do not

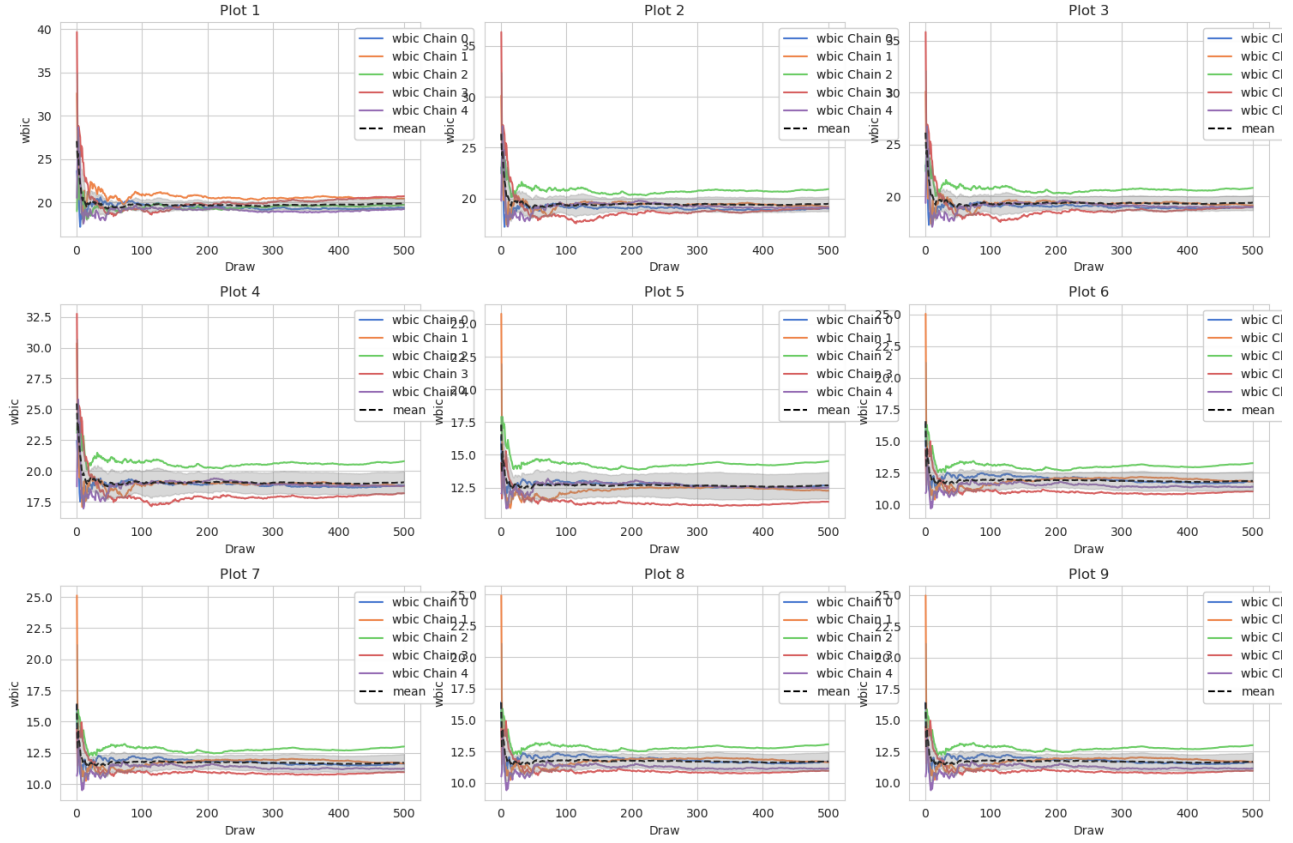


Figure B.2: WBIC trace plots for several snapshots of the total checkpoints for the toy model

convergence at any checkpoint in either of the models. However, the Gelman-Rubin statistic is meant to be computed directly on the samples which we do not currently have access to, and hence was instead computed on the WBIC values across the different chains for each LLC estimation procedure across the checkpoints. Therefore we do not necessarily conclude that the disentangled model's LLC estimates have failed to converge but we can be reasonably sure from inspecting the values for the toy model that its LLC estimates have indeed not converged at any of the checkpoints.

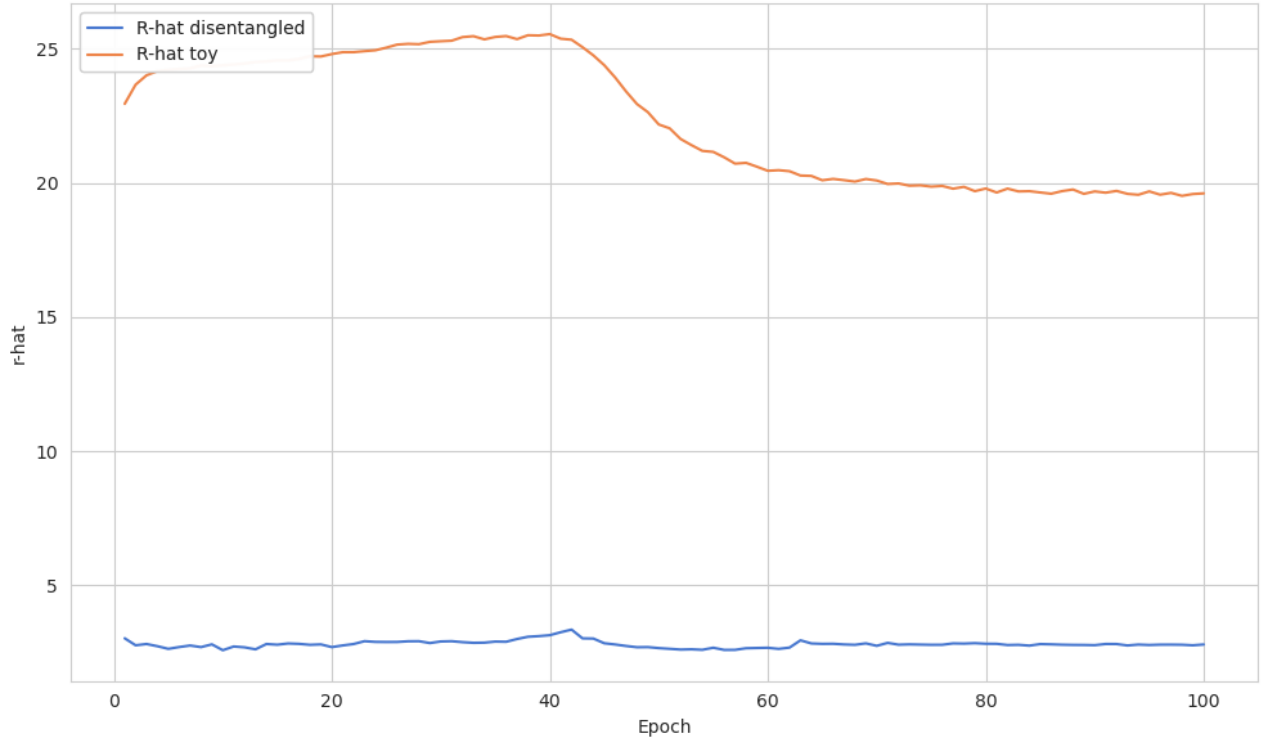


Figure B.3: Rhat for LLC estimate toy and disentangled models

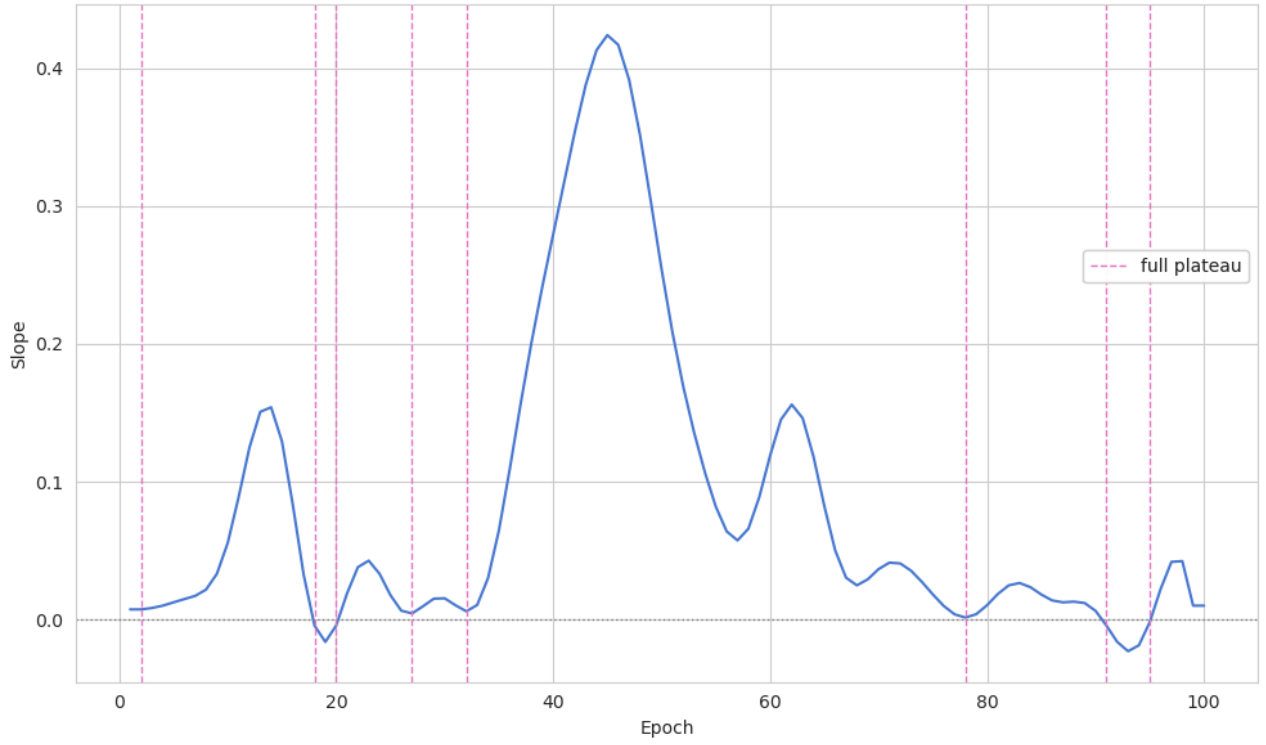


Figure B.4: Slope of RLCT for full model with developmental stages.

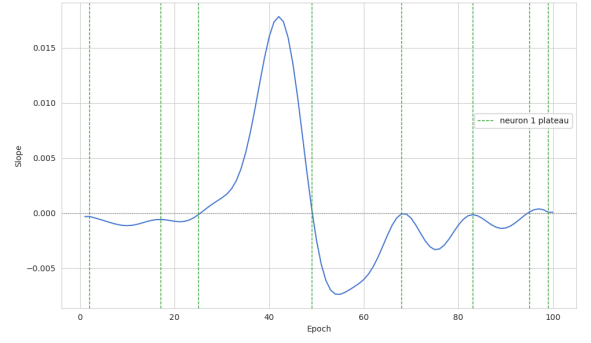
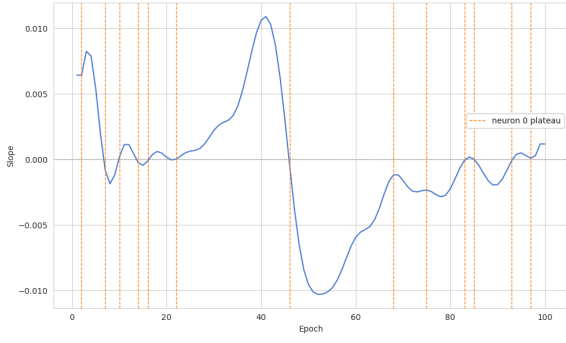


Figure B.5: Slopes with developmental stages for neurons 0 and 1.

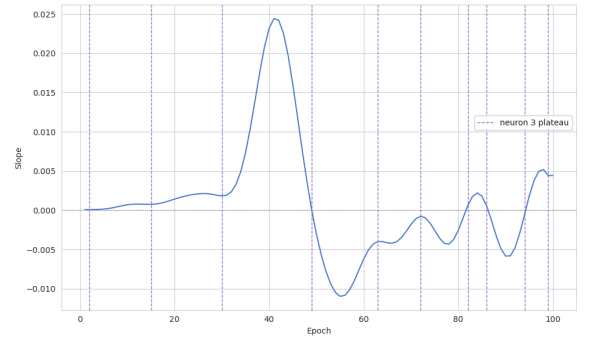
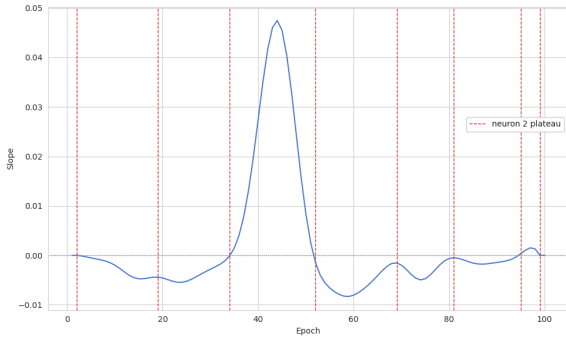


Figure B.6: Slopes with developmental stages for neurons 2 and 3.

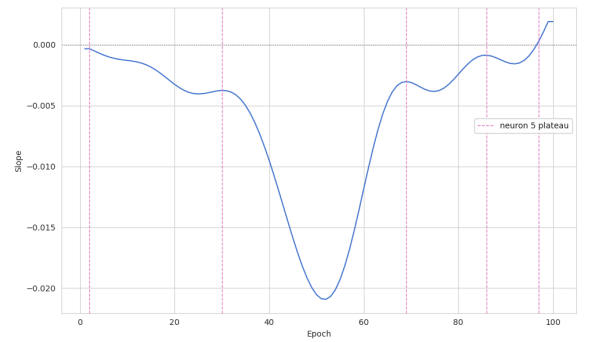
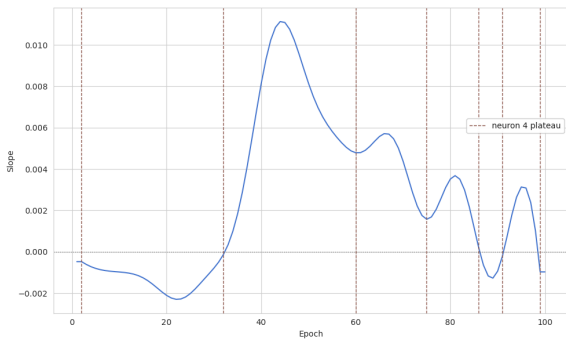


Figure B.7: Slopes with developmental stages for neurons 4 and 5.